



UNIVERSITY OF PISA
MASTER'S DEGREE IN COMPUTER SCIENCE,
ARTIFICIAL INTELLIGENCE

**Computational Mathematics for Learning &
Data Analysis (646AA)**

Group number: 63
Project number: 25 ML

Members:
Matthew Bernardi
Gabriele Marsili

ACADEMIC YEAR: 2025/2026

Project statement.

25. (P) is so-called *extreme learning*, i.e., a neural network with one hidden layer, $y = w\sigma(W_1x)$, where the weight matrix for the hidden layer W_1 is a fixed random matrix, $\sigma(\cdot)$ is an elementwise activation function of your choice, and the output weight vector w is chosen by solving a least-squares problem with L_1 regularization $\arg \min_w f(w)$, with $f(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|w\|_1$.

(A1) is *iteratively reweighted least squares*: i.e., an iteration where you solve at each step the least-squares problem $w_{k+1} = \arg \min_w f_k(w)$, $f_k(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|W_k w\|_2^2$ where W_k is the diagonal matrix with entries $(W_k)_{ii} = |(w_k)_i|^{-1/2}$.

(A2) is an algorithm of the class of the *deflected subgradient methods*.

Contents

1	Introduction to the problem	2
1.1	Extreme Learning Machine	2
1.2	Least-Squares Problem	2
1.3	Regularization Techniques	4
1.3.1	L_1 regularization	4
1.4	Objective Function Analysis	4
1.4.1	Convexity	4
1.4.2	Non-smoothness	5
1.4.3	Optimality conditions	5
2	Algorithm 1: Iteratively Reweighted Least Squares	6
2.1	The core idea: a strict upper bound for the L_1 norm	6
2.2	The Majorization-Minimization interpretation	7
2.3	The weighted least-squares subproblem	8
2.4	The complete algorithm	9
2.5	Convergence analysis	9
2.6	Computational cost	11
2.7	Expected behavior on our problem	11
3	Algorithm 2: Deflected Subgradient Method	13
3.1	Why the plain subgradient method is inadequate	13
3.1.1	The deflection mechanism	13
3.2	Convergence framework: balancing deflection and stepsize	14
3.2.1	Optimal deflection parameter	14
3.3	The target-level mechanism	15
3.4	The complete algorithm	15
3.4.1	Parameter calibration for ELM LASSO	15
3.5	Application to the ELM LASSO problem	18
3.5.1	Subgradient computation for ELM LASSO	18
3.5.2	Convergence analysis and hypothesis verification for ELM LASSO	18
3.6	Expected practical behavior	20

4	Algorithm Comparison	22
4.1	Core strategy: how each algorithm handles non-smoothness . . .	22
4.2	Convergence behaviour	22
4.2.1	Monotonicity	22
4.2.2	Rate	22
4.2.3	Convergence on the ELM problem	23
4.3	Computational cost	23
4.3.1	Per-iteration cost	23
4.3.2	Total cost	24
4.4	Solution quality	24
4.4.1	Sparsity	24
4.4.2	Accuracy	24
4.5	Comparison summary for the ELM problem	24
5	Experimental Results	26
5.1	Experimental setup and datasets	26
5.1.1	Datasets	27
5.2	Convergence and agreement with theory	27
5.3	Cold versus warm start	30
5.3.1	SGPTL beyond the submitted iteration budget	32
5.4	Hyperparameter tuning and technique choices	33
5.4.1	IRLS: ε_{thr} and λ_{LASSO}	33
5.4.2	IRLS: inner solver (Cholesky vs. CG)	34
5.4.3	SGPTL: the deflection floor $\gamma_{\min}$	34
5.4.4	SGPTL: δ_0 and ρ	36
5.4.5	SGPTL: δ_0 against the f^* -based scale	37
5.5	IRLS versus SGPTL	37
5.5.1	Solution quality and sparsity	38
5.5.2	Scalability	38
5.5.3	Iterations to a target accuracy	39
5.6	Validation on real datasets	40
5.7	Initial design and corrections	42
6	Conclusions	44
A	Full δ_0 family sweep	46
B	IRLS warm vs cold start on the synthetic instance	49

Chapter 1

Introduction to the problem

1.1 Extreme Learning Machine

Extreme Learning Machine (ELM) [15] is a machine learning model implemented as a neural network with a single hidden layer, in which:

- $\mathbf{W}_1 \in \mathbb{R}^{H \times N}$ is the input-to-hidden weight matrix (where H is the number of nodes in the hidden layer and N is the number of input nodes) that is **randomly generated** and **static** (does not change during training) [15].
- $\sigma(\cdot)$ is the element-wise (nonlinear) activation function applied to the $\mathbf{W}_1 \mathbf{x} \in \mathbb{R}^H$ vector, where $\mathbf{x} \in \mathbb{R}^N$ is the input vector. The choice of using an activation function for the hidden layer results, under mild assumptions and with high probability, in the hidden layer feature matrix with full rank when $M \geq H$, since random projections through a nonlinear function produce linearly independent features.
- $\mathbf{w} \in \mathbb{R}^H$ is the hidden-to-output weight vector.
- $\hat{y} = \mathbf{w}^T \sigma(\mathbf{W}_1 \mathbf{x}) \in \mathbb{R}$ is the prediction of the model.

Once $\mathbf{W}_1$ is fixed, knowing that $\mathbf{X}_{\text{origin}} \in \mathbb{R}^{M \times N}$ is the matrix of all the inputs, the hidden-layer activations $\mathbf{X} = \sigma(\mathbf{X}_{\text{origin}} \mathbf{W}_1^T)$ are constant, and finding $\mathbf{w}$ reduces to solving the Least-Squares problem described in the following section.

1.2 Least-Squares Problem

The definition of the Least Squares Problem [6] in the ELM case is the following:

$$\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 \quad (1.1)$$

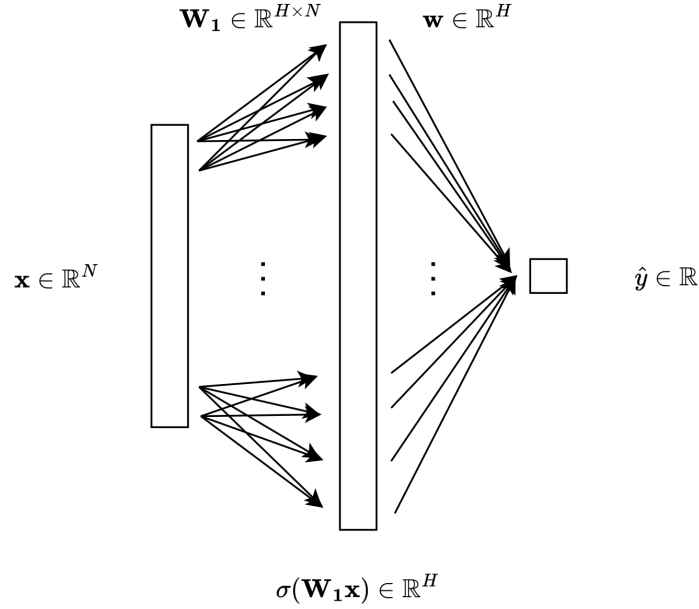


Figure 1.1: Visual representation of an Extreme Learning Machine model.

The Least-Squares problem always admits at least one solution, and the solution is unique if and only if the matrix $\mathbf{X}$ has full column rank. In that case, we can write the optimization problem as follows:

$$\min_{\mathbf{w}} \frac{1}{2} \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{y}^T \mathbf{X} \mathbf{w} + \frac{1}{2} \mathbf{y}^T \mathbf{y} \quad (1.2)$$

since the gradient of this function is $\mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{X}^T \mathbf{y}$, the Hessian matrix is $\mathbf{X}^T \mathbf{X} \succ 0$, which means that the function is strictly convex. In this case, the minimum exists and is unique, and can be found solving the equation:

$$\mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{X}^T \mathbf{y} = 0$$

where $\mathbf{X}^T \mathbf{X}$ is square invertible since it's positive definite. The unique solution is $\mathbf{w}_0 = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$. However, adding the L_1 regularization term (Section 1.3) destroys the closed-form structure, requiring iterative methods.

1.3 Regularization Techniques

1.3.1 L_1 regularization

When adding the L_1 regularization term to the Least-Squares, the equation is the following:

$$\min_{\mathbf{w}} f(\mathbf{w}) = \frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda_{\text{LASSO}} \sum_i^H |w_i| \quad (1.3)$$

This form is particularly nice since it pushes the weights $w_i \rightarrow 0$, but the price we have to pay is the non-differentiability of the function where $w_i = 0$, which prevents direct use of gradient-based methods.

Because of that, we can use the IRLS algorithm (2) and the Deflected Sub-gradient Methods (3).

1.4 Objective Function Analysis

Let's break down the function to minimize in two parts:

$$\min_{\mathbf{w}} f(\mathbf{w}) = \underbrace{\frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2}_{g(\mathbf{w})} + \underbrace{\lambda_{\text{LASSO}} \|\mathbf{w}\|_1}_{h(\mathbf{w})} \quad (1.4)$$

Let's analyse the mathematical properties of $f(\mathbf{w})$.

1.4.1 Convexity

The mathematical properties of the objective function $f(\mathbf{w})$ are heavily dependent on the matrix $\mathbf{X}$.

Proposition 1.1. *If $\mathbf{X} \in \mathbb{R}^{M \times H}$ has full column rank, then $f(\mathbf{w}) = \frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda_{\text{LASSO}} \|\mathbf{w}\|_1$ is strictly convex and admits a unique global minimizer $\mathbf{w}^*$ [2, p. 73].*

Proof. Strict convexity. Since $\mathbf{X}$ has full column rank, for any $\mathbf{v} \neq \mathbf{0}$ we have $\mathbf{X}\mathbf{v} \neq \mathbf{0}$, so $\mathbf{v}^T (\mathbf{X}^T \mathbf{X}) \mathbf{v} = \|\mathbf{X}\mathbf{v}\|_2^2 > 0$. Hence the Hessian of g is $\nabla^2 g = \mathbf{X}^T \mathbf{X} \succ 0$, making g strictly convex. h is convex as a non-negative multiple of a norm. The sum of a strictly convex function and a convex function is strictly convex [12, Slides 21–24], so f is strictly convex.

Existence and uniqueness of the minimizer. Since $\mathbf{X}^T \mathbf{X} \succ 0$, g is coercive: $g(\mathbf{w}) \rightarrow +\infty$ when $\|\mathbf{w}\| \rightarrow \infty$. Because $h \geq 0$, $f = g + h$ is also coercive. This means the sub-level set $\{\mathbf{w} : f(\mathbf{w}) \leq f(\mathbf{0})\}$ is compact, and, since f is continuous, it has its minimum on this set. Strict convexity ensures that there is a single minimizer: if $\mathbf{w}_1 \neq \mathbf{w}_2$ both minimized f , the midpoint $\frac{\mathbf{w}_1 + \mathbf{w}_2}{2}$ would give a strictly smaller value, a contradiction. $\square$

1.4.2 Non-smoothness

Unlike the quadratic term, the L_1 penalty is **not differentiable** [10, Slide 4] at any point where some component $w_i = 0$, so f is non-smooth. At points where all $w_i \neq 0$, the gradient exists and is:

$$\nabla f(\mathbf{w}) = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda_{\text{LASSO}} \text{sign}(\mathbf{w})$$

where $\text{sign}(\mathbf{w})$ is applied element-wise. At points where some $w_i = 0$, one must reason in terms of *subgradients* (see Chapter 2).

1.4.3 Optimality conditions

The optimality condition is expressed using the **subdifferential** [10, Slide 5]: $\mathbf{w}^*$ is a global minimizer for the problem if and only if:

$$0 \in \partial f(\mathbf{w}^*) = \mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + \lambda_{\text{LASSO}} \partial \|\mathbf{w}^*\|_1$$

where the subdifferential of the L_1 norm is given component-wise:

$$[\partial \|\mathbf{w}\|_1]_i = \begin{cases} \{+1\} & \text{if } w_i > 0, \\ \{-1\} & \text{if } w_i < 0, \\ [-1, +1] & \text{if } w_i = 0. \end{cases}$$

Writing it in a component-wise form: for each $i = 1, \dots, H$

$$[\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y})]_i + \lambda_{\text{LASSO}} s_i = 0 \quad s_i \in \partial |w_i^*| \quad (1.5)$$

Chapter 2

Algorithm 1: Iteratively Reweighted Least Squares

Iteratively Reweighted Least Squares (IRLS) [8] is a classical algorithm for solving optimization problems involving non-smooth objective functions of the form of a p -norm. Its core idea is to handle non-smoothness by solving a sequence of *weighted* least-squares problems, each of which is smooth and admits a closed-form solution.

2.1 The core idea: a strict upper bound for the L_1 norm

The difficulty in (1.4) is the L_1 term: convex but non-differentiable in every point where $w_i = 0$ for at least an index i . To replace it with a smooth surrogate function, we use the inequality $(|w_i| - |(w_k)_i|)^2 \geq 0$, which, after expansion and division by $2|(w_k)_i|$, gives the majorization:

$$|w_i| \leq \frac{w_i^2}{2|(w_k)_i|} + \frac{|(w_k)_i|}{2} \quad (w_k)_i \neq 0 \quad (2.1)$$

Summing over $i = 1, \dots, H$ returns the bound on the L_1 norm

$$\|\mathbf{w}\|_1 \leq \frac{1}{2} \mathbf{w}^T \mathbf{W}_k^T \mathbf{W}_k \mathbf{w} + \frac{1}{2} \|\mathbf{w}_k\|_1 \quad (2.2)$$

where $\mathbf{W}_k \in \mathbb{R}^{H \times H}$ is a diagonal matrix with entries:

$$(\mathbf{W}_k)_{ii} = |(w_k)_i|^{-1/2} \quad (2.3)$$

This bounds the non-smooth L_1 term using a *smooth, quadratic* expression $\frac{1}{2} \mathbf{w}^T \mathbf{W}_k^T \mathbf{W}_k \mathbf{w}$, plus a constant term that depends only on the current iteration $\mathbf{w}_k$. $\mathbf{W}_k^T \mathbf{W}_k$ represents a sort of *personalized penalization* of each component

of $\mathbf{w}$ [5]: components with small magnitude at iteration k receive a large weight $|(w_k)_i|^{-1/2}$ and are strongly pushed toward zero, while components with large magnitude receive a small weight and are left relatively free. This *reweighting* mechanism is what drives IRLS to produce sparse iterates.

Handling near-zero components. Equation (2.3) becomes numerically unstable when $(w_k)_i \approx 0$, since $|(w_k)_i|^{-1/2}$ can blow up. To prevent division by (near) zero, a **safety threshold** $\varepsilon_{\text{thr}} > 0$ is introduced:

$$(\mathbf{W}_k)_{ii} = \left(\max(|(w_k)_i|, \varepsilon_{\text{thr}}) \right)^{-1/2} \quad (2.4)$$

Typical values are $\varepsilon_{\text{thr}} \in [10^{-6}, 10^{-10}]$.

2.2 The Majorization-Minimization interpretation

IRLS fits inside the Majorization-Minimization (MM) framework [16]. Instead of minimizing f directly, at each iteration k we build a *surrogate* $Q(\mathbf{w}, \mathbf{w}_k)$ function that:

1. Majorizes f : $Q(\mathbf{w}, \mathbf{w}_k) \geq f(\mathbf{w})$ for all $\mathbf{w}$
2. Touches f at $\mathbf{w}_k$: $Q(\mathbf{w}_k, \mathbf{w}_k) = f(\mathbf{w}_k)$
3. Is smooth, so that it can be minimized in closed form

Plugging the quadratic upper bound (2.2) into (1.4) gives the following surrogate:

$$Q(\mathbf{w}, \mathbf{w}_k) = \frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2} \|\mathbf{W}_k \mathbf{w}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2} \|\mathbf{w}_k\|_1 \quad (2.5)$$

We need to check that the properties are satisfied:

1. Majorization property: follows from (2.1)
2. Touching property: at $\mathbf{w} = \mathbf{w}_k$, we get the following Q function

$$Q(\mathbf{w}_k, \mathbf{w}_k) = \frac{1}{2} \|\mathbf{X}\mathbf{w}_k - \mathbf{y}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2} \|\mathbf{w}_k\|_1 + \frac{\lambda_{\text{LASSO}}}{2} \|\mathbf{w}_k\|_1 = f(\mathbf{w}_k)$$

3. **Smoothness property:** The surrogate $Q(\mathbf{w}, \mathbf{w}_k)$ is a function of $\mathbf{w}$, since $\mathbf{w}_k$ is fixed (constant). Its gradient is

$$\nabla_{\mathbf{w}} Q(\mathbf{w}, \mathbf{w}_k) = (\mathbf{X}^T \mathbf{X} + \lambda_{\text{LASSO}} \mathbf{W}_k^T \mathbf{W}_k) \mathbf{w} - \mathbf{X}^T \mathbf{y}$$

Since it is an affine function of $\mathbf{w}$, the Hessian is a constant matrix. Because of that, Q is infinitely differentiable (C^∞), and also globally L -smooth (with a Lipschitz continuous gradient).

Taking $\mathbf{w}_{k+1} = \arg \min_{\mathbf{w}} Q(\mathbf{w}, \mathbf{w}_k)$, the first two MM properties combined give $f(\mathbf{w}_{k+1}) \leq Q(\mathbf{w}_{k+1}, \mathbf{w}_k) \leq Q(\mathbf{w}_k, \mathbf{w}_k) = f(\mathbf{w}_k)$, so the sequence $\{f(\mathbf{w}_k)\}$ is non-increasing.

2.3 The weighted least-squares subproblem

To find $\mathbf{w}_{k+1}$, we must minimize the surrogate function $Q(\mathbf{w}, \mathbf{w}_k)$ with respect to $\mathbf{w}$:

$$\mathbf{w}_{k+1} = \arg \min_{\mathbf{w}} \left(\frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2} \|\mathbf{W}_k \mathbf{w}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2} \|\mathbf{w}_k\|_1 \right)$$

Since the term $\frac{\lambda_{\text{LASSO}}}{2} \|\mathbf{w}_k\|_1$ is constant with respect to $\mathbf{w}$, it does not affect the optimization and can be dropped. The minimization problem simplifies to a standard **weighted least-squares problem with L_2 regularization**:

$$\mathbf{w}_{k+1} = \arg \min_{\mathbf{w}} \left(\frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda_{\text{IRLS}} \|\mathbf{W}_k \mathbf{w}\|_2^2 \right) \quad (2.6)$$

using:

$$\lambda_{\text{IRLS}} = \frac{1}{2} \lambda_{\text{LASSO}} \quad (2.7)$$

Proposition 2.1 (Weighted normal equation solution). *The unique solution of (2.6) satisfies:*

$$(\mathbf{X}^T \mathbf{X} + 2\lambda_{\text{IRLS}} \mathbf{W}_k^T \mathbf{W}_k) \mathbf{w}_{k+1} = \mathbf{X}^T \mathbf{y} \quad (2.8)$$

Proof. The gradient of the objective function (2.6) is:

$$\nabla f_k(\mathbf{w}) = \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y}) + 2\lambda_{\text{IRLS}} \mathbf{W}_k^T \mathbf{W}_k \mathbf{w}$$

Setting this to zero results in (2.8). The coefficient matrix $\mathbf{X}^T \mathbf{X} + 2\lambda_{\text{IRLS}} \mathbf{W}_k^T \mathbf{W}_k$ is symmetric positive definite (it is the sum of $\mathbf{X}^T \mathbf{X} \succeq 0$ and $2\lambda_{\text{IRLS}} \mathbf{W}_k^T \mathbf{W}_k \succ 0$ since $\varepsilon_{\text{thr}} > 0$), so the system has a unique solution. $\square$

Since $\mathbf{W}_k$ is diagonal, defining $\mathbf{Q}_k = \mathbf{X}^T \mathbf{X} + 2\lambda_{\text{IRLS}} \mathbf{W}_k^T \mathbf{W}_k$ and $\mathbf{b} = \mathbf{X}^T \mathbf{y}$, equation (2.8) reduces to:

$$\mathbf{Q}_k \mathbf{w}_{k+1} = \mathbf{b} \quad (2.9)$$

This is an $H \times H$ symmetric positive definite linear system, which can be solved efficiently at each iteration. Note that $\mathbf{A} = \mathbf{X}^T \mathbf{X}$ and $\mathbf{b} = \mathbf{X}^T \mathbf{y}$ can be pre-computed once before the loop, so only the diagonal $2\lambda_{\text{IRLS}} \mathbf{W}_k^T \mathbf{W}_k$ is updated at each iteration.

Solving the linear system. Because $\mathbf{Q}_k$ is SPD, two efficient methods can be used:

- **Cholesky factorization:** [18] computes $\mathbf{Q}_k = \mathbf{L}\mathbf{L}^T$ in $O(H^3/3)$ operations, then solves two triangular systems in $O(H^2)$. This is the direct method of choice for moderate-sized problems.
- **Conjugate Gradient (CG):** [18] an iterative method that converges in at most H steps in exact arithmetic, with convergence rate depending on the condition number $\kappa(\mathbf{Q}_k)$. Preferable for large-scale problems where H is too large for Cholesky.

2.4 The complete algorithm

Algorithm 1 Iteratively Reweighted Least Squares (IRLS)

Require: $\mathbf{X} \in \mathbb{R}^{M \times H}$, $\mathbf{y} \in \mathbb{R}^M$, $\lambda_{\text{LASSO}} > 0$, $\varepsilon_{\text{thr}} > 0$, $\varepsilon_{\text{stop}} > 0$, k_{max}

- 1: Pre-compute $\mathbf{A} \leftarrow \mathbf{X}^T \mathbf{X}$, $\mathbf{b} \leftarrow \mathbf{X}^T \mathbf{y}$
- 2: Define $\lambda_{\text{IRLS}} \leftarrow \frac{1}{2} \lambda_{\text{LASSO}}$
- 3: Initialize $\mathbf{w}_0 \leftarrow \mathbf{A}^{-1} \mathbf{b}$ (ordinary least-squares solution)
- 4: **for** $k = 0, 1, 2, \dots, k_{\text{max}}$ **do**
- 5: Compute weights: $(\mathbf{W}_k)_{ii} \leftarrow (\max(|(w_k)_i|, \varepsilon_{\text{thr}}))^{-1/2}$ for all i
- 6: Assemble system: $\mathbf{Q}_k \leftarrow \mathbf{A} + 2\lambda_{\text{IRLS}} \mathbf{W}_k^T \mathbf{W}_k$
- 7: Solve: $\mathbf{Q}_k \mathbf{w}_{k+1} = \mathbf{b}$ (via *Cholesky* or *CG*)
- 8: **if** $\|\mathbf{w}_{k+1} - \mathbf{w}_k\| / \max(1, \|\mathbf{w}_k\|) < \varepsilon_{\text{stop}}$ **then**
- 9: **break** (convergence reached)
- 10: **end if**
- 11: **end for**
- 12: **return** $\mathbf{w}_{k+1}$

Initialization. We initialize $\mathbf{w}_0$ with the unregularized least-squares solution $\mathbf{A}^{-1} \mathbf{b}$ (**warm start**); the weights $(\mathbf{W}_0)_{ii} = |(w_0)_i|^{-1/2}$ are well defined without having to lean on ε_{thr} at the first step. A **cold start** with $\mathbf{w}_0 = \mathbf{0}$ would instead force every weight onto the threshold and slow the first iterations down. In the implementation we add a vanishing ridge $10^{-12} \mathbf{I}$ to $\mathbf{A}$ before this first solve: numerically indistinguishable from $\mathbf{A}^{-1} \mathbf{b}$ when $\mathbf{A} \succ 0$, it keeps the warm start well-posed if $\mathbf{X}$ is rank-deficient.

Stopping criterion. The algorithm ends when the relative change between successive iterations goes below $\varepsilon_{\text{stop}}$:

$$\frac{\|\mathbf{w}_{k+1} - \mathbf{w}_k\|}{\max(1, \|\mathbf{w}_k\|)} < \varepsilon_{\text{stop}}$$

where the $\max(1, \|\mathbf{w}_k\|)$ normalization makes the test scale-invariant and prevents false triggers when $\|\mathbf{w}_k\|$ is small.

2.5 Convergence analysis

Monotone decrease. As said in Section 2.2, IRLS monotonically decreases the function. Since f is bounded below by zero, the sequence $\{f(\mathbf{w}_k)\}$ converges.

Theorem 2.2 (Fixed-point consistency). *If the sequence $\{\mathbf{w}_k\}$ generated by IRLS converges to a point $\mathbf{w}^*$ such that $|(w^*)_i| > \varepsilon_{\text{thr}}$ for all i , then $\mathbf{w}^*$ satisfies 1.5.*

Proof. At a fixed point $\mathbf{w}_k = \mathbf{w}_{k+1} = \mathbf{w}^*$, (2.8) gives:

$$\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + 2\lambda_{\text{IRLS}} \mathbf{W}_*^T \mathbf{W}_* \mathbf{w}^* = \mathbf{0}$$

Since $|(w^*)_i| > \varepsilon_{\text{thr}}$ for all i , the threshold is inactive and $(\mathbf{W}_*^T \mathbf{W}_*)_ii = |(w^*)_i|^{-1}$. The i -th component of the regularizer term is:

$$(\mathbf{W}_*^T \mathbf{W}_* \mathbf{w}^*)_i = \frac{(w^*)_i}{|(w^*)_i|} = \text{sign}((w^*)_i)$$

Using (2.7):

$$\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + \lambda_{\text{LASSO}} \text{sign}(\mathbf{w}^*) = \mathbf{0}$$

Since $|(w^*)_i| \neq 0$ for all i , the L_1 norm is differentiable at $\mathbf{w}^*$, so $\partial\|\mathbf{w}^*\|_1 = \text{sign}(\mathbf{w}^*)$ [10]. By the sum rule for subdifferentials [12], the equation above is exactly $\mathbf{0} \in \partial f(\mathbf{w}^*)$, the optimality condition in (1.5) is met. $\square$

Convergence rate. The convergence rate of the algorithm is linear: the error $\|\mathbf{w}_k - \mathbf{w}^*\|$ shrinks by a constant factor each iteration, set by the conditioning number of the matrix $\mathbf{Q}_k = \mathbf{X}^T \mathbf{X} + 2\lambda_{\text{IRLS}} \mathbf{W}_k^T \mathbf{W}_k$. The second term also acts as a Tikhonov preconditioner to improve the conditioning of the matrix. This analysis, observed by us empirically, is consistent with Daubechies et al.'s analysis [8].

Verification of hypotheses for the ELM problem. The two assumptions of Theorem 2.2 are satisfied for (1.4) as follows.

1. *Convergence of the sequence.* Under the assumption that $\mathbf{X}$ has full column rank, f is strictly convex and admits a unique minimizer $\mathbf{w}^*$ (Proposition 1.1). The convergence $\mathbf{w}_k \rightarrow \mathbf{w}^*$ follows in four steps: IRLS monotonically decreases f_k converging to $f \geq 0$. f is coercive (Proposition 1.1), so every sublevel set is compact and $\{\mathbf{w}_k\}$ has at least one accumulation point $\bar{\mathbf{w}}$. Continuity of the surrogate $Q(\cdot, \mathbf{w}_k)$ and the MM descent property $Q(\mathbf{w}_{k+1}, \mathbf{w}_k) \leq Q(\mathbf{w}_k, \mathbf{w}_k) = f(\mathbf{w}_k)$ force $\bar{\mathbf{w}}$ to be a fixed point of the IRLS iteration. By the Theorem 2.2, every such fixed point satisfying $|(\bar{w})_i| > \varepsilon_{\text{thr}}$ is a global minimizer, so $\bar{\mathbf{w}} = \mathbf{w}^*$ by uniqueness. Since every accumulation point coincides with $\mathbf{w}^*$, the full sequence converges.
2. *Non-zero components at convergence.* The hypothesis $|(w^*)_i| > \varepsilon_{\text{thr}}$ holds for the non-zero components of the sparse solution. Components that collapse to zero within ε_{thr} are effectively frozen by the weight clipping (2.4) and drop out of the optimality condition (1.5), which is exactly the behaviour we want, because these components correspond to features that the L_1 penalty has marked as inactive.

2.6 Computational cost

The computational cost of each iteration of IRLS is the following:

- **Weight update** $\mathbf{W}_k$: $O(H)$ simply iterating over the components of $\mathbf{w}_k$.
- **Matrix update** $\mathbf{Q}_k = \mathbf{A} + 2\lambda_{\text{IRLS}} \mathbf{W}_k^T \mathbf{W}_k$: $O(H)$, since $\mathbf{A} = \mathbf{X}^T \mathbf{X}$ is pre-computed and only the diagonal changes.
- **Linear system solve** $\mathbf{Q}_k \mathbf{w}_{k+1} = \mathbf{b}$: $O(H^3/3)$ with Cholesky, or $O(pH^2)$ with CG for p CG steps (in the following we assume using Cholesky).

Also, we need $O(MH^2)$ for the $\mathbf{A}$ matrix computation and $O(MH)$ for the $\mathbf{b}$ vector creation, and, in case of warm start, we need the OLS solution $\mathbf{w}_0 \leftarrow \mathbf{A}^{-1} \mathbf{b}$, computed in $O(H^3)$. The dominant cost per iteration is the linear system solve: $O(H^3)$ with Cholesky. However, the total number of IRLS iterations is typically small (10–50), and the pre-computation costs get amortized over all iterations. The overall cost is therefore:

$$\underbrace{O(MH^2) + O(MH) + O(H^3)}_{\text{pre-computation}} + \underbrace{k_{\text{IRLS}} \cdot O(H^3)}_{\text{iterations cost}}$$

For problems where $H \ll M$, the pre-computation dominates and IRLS is very efficient.

We can also consider using a cold start initialization for the $\mathbf{w}_0$ vector, dropping the $O(H^3)$ term (same cost of a single loop iteration) in the pre-computation phase. That would result in a much slower convergence, requiring more than one additional loop iteration to achieve the same level of precision as the warm start technique, especially considering that, at the beginning, the changes in the $\mathbf{w}_k$ vector are very small. For these reasons, the cold start is considered a bad practice for this algorithm.

2.7 Expected behavior on our problem

From the analysis above we expect the following behaviour on the ELM LASSO problem. The objective $f(\mathbf{w}_k)$ decreases at every iteration, with no oscillations, which makes IRLS easy to monitor and to debug: a single non-decreasing step signals an error. On a semilog plot of $f(\mathbf{w}_k) - f^*$ against k , we expect an approximately straight line, with a slope dependent on the conditioning of $\mathbf{Q}_k$ and the safety threshold ε_{thr} .

Sparsity is produced dynamically: components $(w_k)_i$ that start out small get assigned increasingly large weights $|(w_k)_i|^{-1}$, which pushes them further toward zero at the next iteration. The stable outcome is a weight vector with many components inside an ε_{thr} -ball of zero. We have 2 hyperparameters to tune: the regularization strength λ_{LASSO} (the model parameter), and the safety threshold ε_{thr} (to define the numerical stability of the algorithm). For λ_{LASSO} , larger values give sparser solutions but a higher training residual. Also, the

conditioning number of $\mathbf{Q}_k$ also depends on λ_{LASSO} , so the convergence speed moves with it. For ε_{thr} , smaller values approximate the true L_1 better, resulting in more sparse solutions, but pay the price in stability whenever a component goes very close to zero. Setting it 10^{-8} is our default choice during experiments.

Chapter 3

Algorithm 2: Deflected Subgradient Method

In this chapter, we apply the deflected subgradient method with Polyak step and with target level (SGPTL) to the ELM LASSO problem.

3.1 Why the plain subgradient method is inadequate

The ELM LASSO problem (1.3) is nonsmooth due to the ℓ_1 penalty. The plain subgradient method converges on convex nonsmooth problems but has two structural limitations on ELM LASSO. First, the update $\mathbf{w}_{i+1} = \mathbf{w}_i - \alpha_i \mathbf{g}_i$ has no memory [10, Slide 10] and zig-zags on ill-conditioned $\mathbf{X}^T \mathbf{X}$; while it attains the optimal iteration complexity $\Omega(L^2/\varepsilon^2)$ [4, Thm. 3.2]¹, the constant L depends on $\|\mathbf{X}\|_2$ and the sublevel-set radius. Second, a pre-set diminishing-square-summable schedule $\sum_i \alpha_i = \infty$, $\sum_i \alpha_i^2 < \infty$ [10, Slide 10] forces the step to vanish regardless of the actual gap to f^* , so practical progress stalls. We address the first issue with deflection, which injects memory into the search direction, and the second with an adaptive Polyak target-level stepsize in place of a fixed schedule (Section 3.3).

3.1.1 The deflection mechanism

Deflected subgradient methods [10, Slide 15] replace the raw subgradient with a direction that blends the current subgradient with the previous search direction:

$$\mathbf{d}_i = \gamma_i \mathbf{g}_i + (1 - \gamma_i) \mathbf{d}_{i-1} \quad (i \geq 1), \quad \mathbf{d}_0 = \mathbf{g}_0, \quad (3.1)$$

¹From the Polyak bound $\bar{f}_k - f^* \leq L \|\mathbf{w}_0 - \mathbf{w}^*\|/\sqrt{k}$ in [4, Thm. 3.2] one gets $k = O(L^2 \|\mathbf{w}_0 - \mathbf{w}^*\|^2/\varepsilon^2)$, which we write as $O(L^2/\varepsilon^2)$ absorbing the initial radius. The form $O(L/\varepsilon^2)$ that appears in some sources is the same order after rescaling ε by L .

where $\gamma_i \in (0, 1]$ is the deflection parameter. The update is $\mathbf{w}_{i+1} = \mathbf{w}_i - \alpha_i \mathbf{d}_i$; $\gamma_i = 1$ recovers plain subgradient, $\gamma_i < 1$ damps the zig-zag. The method specifies γ_i as the minimiser of $\|\mathbf{d}_i\|^2$ on $[0, 1]$ [10, Slide 15], which maximises the Polyak step α_i since $\|\mathbf{d}_i\|^2$ is its denominator.

3.2 Convergence framework: balancing deflection and stepsize

We use the stepsize-restricted Polyak rule [10, Slide 15]:

$$\alpha_i = \frac{\beta_i (f(\mathbf{w}_i) - f^*)}{\|\mathbf{d}_i\|_2^2}, \quad \beta_i \leq \gamma_i, \quad (3.2)$$

with $\beta_i = \min(\beta, \gamma_i)$ and $\beta = 1$ [10, Slide 14]. Since f^* is unknown, we substitute the target level $f_{\text{ref}} - \delta$ (Section 3.3).

Deflection floor $\gamma_{\min} > 0$. Condition (3.5) of [7] requires (when $\lambda_k \geq 0$) the uniform bound $\beta_k \geq \beta^* > 0$ on the stepsize multiplier — in our notation $\beta_i \geq \beta^*$ for all i , not just pointwise positivity. Since $\beta_i = \min(1, \gamma_i)$, a uniform $\gamma_i \geq \gamma_{\min} \in (0, 1]$ supplies $\beta_i \geq \gamma_{\min}$, i.e. $\beta^* = \gamma_{\min}$. The closed-form greedy (3.3) gives $\gamma_i \in (0, 1]$ at each step but does not bound the sequence away from zero: when consecutive subgradients align near stationarity, the numerator $\|\mathbf{d}_{i-1}\|_2^2 - \langle \mathbf{g}_i, \mathbf{d}_{i-1} \rangle$ of γ^* in (3.4) tends to 0^+ and γ_i decays, dragging β_i and the Polyak step to zero — the iterate freezes. Projecting onto $[\gamma_{\min}, 1]$ with $\gamma_{\min} = 0.05$ supplies the missing uniform bound. A sweep $\gamma_{\min} \in \{0.05, 0.1, 0.2, 0.5\}$ on the convergence instance gives record gaps in $[10^{-7}, 10^{-5}]$, with $\gamma_{\min} = 0.05$ marginally best (Section 5.1).

3.2.1 Optimal deflection parameter

The argmin problem

$$\gamma_i \in \arg \min_{\gamma \in [\gamma_{\min}, 1]} \|\gamma \mathbf{g}_i + (1 - \gamma) \mathbf{d}_{i-1}\|_2^2 \quad (3.3)$$

is quadratic in γ ; expanding $\|\gamma \mathbf{g}_i + (1 - \gamma) \mathbf{d}_{i-1}\|_2^2$ by bilinearity and taking the vertex of the resulting upward parabola gives the closed form (annotated as such in [10, Slide 15]):

$$\gamma^* = \frac{\|\mathbf{d}_{i-1}\|_2^2 - \langle \mathbf{g}_i, \mathbf{d}_{i-1} \rangle}{\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2}, \quad \gamma_i = \min(1, \max(\gamma_{\min}, \gamma^*)). \quad (3.4)$$

The clip onto $[\gamma_{\min}, 1]$ instead of $[0, 1]$ is our addition (Section 3.2); the degenerate case $\mathbf{g}_i = \mathbf{d}_{i-1}$ has vanishing leading coefficient and gives $\mathbf{d}_i = \mathbf{d}_{i-1}$ for any γ , so we set $\gamma_i = 1$ by convention. The first iteration is handled analogously:

with $\mathbf{d}_{-1} = \mathbf{0}$ the formula (3.4) returns $\gamma^* = 0$, which would make $\mathbf{d}_0 = \mathbf{0}$; we set $\gamma_0 = 1$ so that $\mathbf{d}_0 = \mathbf{g}_0$, the plain subgradient first step. In the zig-zag regime $\mathbf{g}_i \approx -\mathbf{d}_{i-1}$ with $\|\mathbf{g}_i\| \approx \|\mathbf{d}_{i-1}\|$, the formula gives $\gamma^* \approx 1/2$ and the oscillatory component of $\mathbf{d}_i$ cancels.

3.3 The target-level mechanism

The Polyak stepsize [10, Slide 12] requires f^* . The fundamental inequality [10, Slide 10] is

$$\|\mathbf{w}_{i+1} - \mathbf{w}^*\|_2^2 \leq \|\mathbf{w}_i - \mathbf{w}^*\|_2^2 - 2\alpha_i(f(\mathbf{w}_i) - f^*) + (\alpha_i)^2 \|\mathbf{g}_i\|_2^2, \quad (3.5)$$

minimized at $\alpha_i^* = (f(\mathbf{w}_i) - f^*) / \|\mathbf{g}_i\|_2^2$ [10, Slide 12]. SGPTL replaces f^* with the target level $f_{\text{ref}} - \delta$ [10, Slide 14]: f_{ref} is the best value found so far, $\delta > 0$ an estimate of the residual gap.

A failure mode of this substitution is $f_{\text{ref}} - \delta < f^*$, in which case $f(\mathbf{w}_{i+1}) \geq f^* > f_{\text{ref}} - \delta/2$ for every iterate and the sufficient-descent test $f(\mathbf{w}_{i+1}) \leq f_{\text{ref}} - \delta/2$ is unsatisfiable: f_{ref} never updates and the algorithm silently stalls. The mechanism guards against this through an internal patience counter $r_i = \sum \alpha_j \|\mathbf{d}_j\|$, the displacement accumulated since the last f_{ref} update. When r_i exceeds a threshold R without sufficient descent firing, δ is contracted to $\rho\delta$ with $\rho \in (0, 1)$ and r is reset. Repeated contractions drive $\delta \rightarrow 0$, eventually restoring $f_{\text{ref}} - \delta \geq f^*$ and unblocking the sufficient-descent test.

3.4 The complete algorithm

Algorithm 2 presents the full Deflected Subgradient Method with target level for ELM LASSO.

Numerical safeguards. Three pure floating-point guards lie outside the pseudocode. (i) If $\|\mathbf{d}_i\|^2 < 10^{-30}$ we terminate. (ii) If $f(\mathbf{w}_i) - (f_{\text{ref}} - \delta) \leq 0$ we set $\alpha_i = 0$ (condition (3.5) of [7]) and trigger the sufficient-descent update at $\mathbf{w}_i$, since $f(\mathbf{w}_i) \leq f_{\text{ref}} - \delta/2$. (iii) If $\mathbf{w}_{i+1}$ has a non-finite component we skip the iteration without touching $(\delta, f_{\text{ref}}, r)$.

3.4.1 Parameter calibration for ELM LASSO

SGPTL has five parameters: β , $\gamma_{\min}$, δ_0 , ρ , R . Each is discussed below with the choice we adopt for ELM LASSO and its rationale.

Cold vs. warm start. Theorem 3.1 places no condition on $\mathbf{w}_0$, so we keep the algorithm's default cold start $\mathbf{w}_0 = \mathbf{0}$ and additionally support the OLS warm start $(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$, which lets us run IRLS and SGPTL from the same point when comparing them (Chapter 4). Measured against an independent reference f^* , the cold start is the one that genuinely optimizes, closing most of the initial

Algorithm 2 Deflected Subgradient with Target Level (SGPTL) for ELM LASSO [10, Slide 15]

Require: f (ELM LASSO objective); $i_{\max}$; $\beta \in (0, 1]$; $\delta_0 > 0$; $R > 0$;
 $\rho \in (0, 1)$; $\gamma_{\min} \in (0, 1]$
1: $\mathbf{w}_0 \leftarrow \mathbf{0}$ (*cold start by default; warm start $(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ is admissible — see §3.6*)
2: $r \leftarrow 0$; $\delta \leftarrow \delta_0$; $f_{\text{ref}} \leftarrow \bar{f} \leftarrow f(\mathbf{w}_0)$
3: **for** $i = 0, 1, \dots, i_{\max}$ **do**
4: Compute $\mathbf{g} \in \partial f(\mathbf{w}_i)$ (*subgradient of LASSO*)
5: **if** $i = 0$ **then** (*no $\mathbf{d}_{-1}$ available*)
6: $\gamma_i \leftarrow 1$; $\mathbf{d}_i \leftarrow \mathbf{g}$
7: **else**
8: Compute γ_i via (3.4) (*optimal deflection, clipped to $[\gamma_{\min}, 1]$*)
9: $\mathbf{d}_i \leftarrow \gamma_i \mathbf{g} + (1 - \gamma_i) \mathbf{d}_{i-1}$ (*deflected direction*)
10: **end if**
11: $\beta_i \leftarrow \min(\beta, \gamma_i)$ (*stepsize-restricted, $\beta = 1$ default; see §3.2*)
12: $\alpha_i \leftarrow \beta_i (f(\mathbf{w}_i) - (f_{\text{ref}} - \delta)) / \|\mathbf{d}_i\|_2^2$ (*Polyak step*)
13: $\mathbf{w}_{i+1} \leftarrow \mathbf{w}_i - \alpha_i \mathbf{d}_i$ (*update*)
14: $\bar{f} \leftarrow \min(\bar{f}, f(\mathbf{w}_{i+1}))$ (*update record*)
15: **if** $f(\mathbf{w}_{i+1}) \leq f_{\text{ref}} - \delta/2$ **then** (*sufficient descent*)
16: $f_{\text{ref}} \leftarrow \bar{f}$; $r \leftarrow 0$
17: **else if** $r > R$ **then** (*travel-distance patience exhausted*)
18: $\delta \leftarrow \delta \rho$; $r \leftarrow 0$
19: **else**
20: $r \leftarrow r + \alpha_i \|\mathbf{d}_i\|_2$
21: **end if**
22: **end for**
23: **return** $\mathbf{w}_{\text{best}}$ (iterate achieving $\bar{f}$)

gap on every instance; the warm start ends lower only on California-ELM, and there because $f(\mathbf{w}_{\text{OLS}})$ already sits close to f^* , leaving SGPTL no room to descend, rather than because the warm start aids the method. Section 5.3 reports the figures and per-instance gaps.

$\beta = 1$. [7, (3.1)] admits $\beta_k \in (0, 1]$. Substituting the Polyak step $\alpha_i = \beta(f(\mathbf{w}_i) - f^*) / \|\mathbf{g}_i\|_2^2$ into the fundamental inequality (3.5) gives a per-iteration reduction in $\|\mathbf{w}_i - \mathbf{w}^*\|_2^2$ proportional to $\beta(2 - \beta)$, a downward parabola maximised at $\beta = 1$; the deflected variant of [10, Slide 15] inherits the property. We fix $\beta = 1$ so that $\beta_i = \min(\beta, \gamma_i)$ is driven by γ_i .

$\gamma_{\min} = 0.05$. Condition (3.5) of [7] requires $\beta^* > 0$ strictly, which the floor realises with $\beta^* = \gamma_{\min}$. The value inside $(0, 1]$ is then a tradeoff: a small $\gamma_{\min}$ keeps the clip from binding on most steps, so the deflected direction stays close to the unconstrained greedy minimiser γ^* of (3.4) and preserves the locally optimal Polyak step, at the cost of a $1/\sqrt{\gamma_{\min}}$ factor in the rate of Theorem 3.1 (equivalently $1/\gamma_{\min}$ in the iteration count); a large $\gamma_{\min}$ shrinks the constant but binds the clip frequently, diluting the memory term in $\mathbf{d}_i$. We adopt $\gamma_{\min} = 0.05$, the smallest value in the sweep $\{0.05, 0.1, 0.2, 0.5\}$ of Chapter 5 at which the clip activates only on the near-stationarity degenerate steps documented in Section 3.2.

$\delta_0 = c f(\mathbf{w}_0)$ **with** $c = 0.1$. [7, Lemma 3.8] requires only $\delta_0 > 0$; we need a problem-dependent scale that does not presuppose f^* . We scale on the objective at the starting point, $\delta_0 = c f(\mathbf{w}_0)$. Since f^* is the global minimum, $f(\mathbf{w}_0) \geq f^*$ for any $\mathbf{w}_0$, so the initial target $f_{\text{ref}} - \delta_0$ is a valid overestimate of the residual gap; if it overshoots, the patience mechanism of Section 3.3 contracts δ until the target is feasible. For the default cold start $\mathbf{w}_0 = \mathbf{0}$ this scale is $f(\mathbf{0}) = \frac{1}{2} \|\mathbf{y}\|_2^2$, an $O(M)$ evaluation that needs no precomputation; a warm start instead pays the $O(MH^2 + H^3)$ to form $\mathbf{w}_{\text{OLS}}$ first.

The multiplier c trades off two failure modes: small c keeps the target $f_{\text{ref}} - \delta$ close to f_{ref} and δ barely contracts, while large c pushes the target below f^* so that only the patience trigger drives progress. Following the moderate values of the target-level literature [3, 13, 1] we set $c = 0.1$. Section 5.4 confirms robustness across $c \in [0.01, 1]$, and as an abstract check (Figure A.1) that this scale stays within a factor of two of the f^* -based scale $\delta_0 = c f^*$ wherever SGPTL is in fact optimising — the latter being usable only as a reference, since f^* is unknown in practice.

$\rho = 0.7$. [7, Lemma 3.8] admits any $\rho \in (0, 1)$. Tradeoff: ρ close to 1 contracts δ slowly, keeping the target too aggressive for a large fraction of the budget; ρ close to 0 contracts before the algorithm has had time to exploit each target level. The target-level literature [3, 13, 1], [10, Slide 14] recommends the moderate range $[0.5, 0.7]$. A smoke-test sweep at $\rho \in \{0.3, 0.5, 0.7, 0.8, 0.9, 0.95\}$ on the synthetic instance and on the two real-data ELM matrices places different ρ as

the per-instance best, but only $\rho = 0.7$ is within a factor of two of the best on every tested instance.

$R = 1$. [7, Lemma 3.8] admits any $R > 0$ and gives no specific value. Tradeoff: R too small contracts δ on every minor stagnation, wasting potential progress between target updates; R too large makes the sufficient-descent test the sole driver and stalls if the target is structurally infeasible. R should therefore sit on the per-iteration travel scale $\alpha_i \|\mathbf{d}_i\|$, which under our other defaults on ELM LASSO with column-normalised $\mathbf{X}$ is of order 10^{-3} . We adopt $R = 1$, so the travel-distance branch fires every $\mathcal{O}(10^3)$ stalled iterations; Section 5.3 reports the empirical contraction counts.

3.5 Application to the ELM LASSO problem

3.5.1 Subgradient computation for ELM LASSO

Section 1.4.3 already gives the subdifferential of the ELM LASSO objective. Algorithm 2 requires a single element $\mathbf{g}_i \in \partial f(\mathbf{w}_i)$ per iteration; we pick the canonical sign-based representative

$$\mathbf{g} = \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda_{\text{LASSO}} \mathbf{s}, \quad s_i = \text{sign}(w_i) \in \partial|w_i|, \quad (3.6)$$

with the convention $\text{sign}(0) = 0$, which is admissible since $0 \in [-1, 1] = \partial|0|$. This choice is not the minimum-norm element of $\partial f(\mathbf{w}_i)$, but Theorem 3.1 only requires admissibility. The event $(\mathbf{w}_i)_j = 0$ exactly is non-generic for a subgradient method — iterates are updated continuously by $\mathbf{w}_{i+1} = \mathbf{w}_i - \alpha_i \mathbf{d}_i$ and never land on a coordinate hyperplane unless $\mathbf{w}_0$ is placed there to begin with; the cold start $\mathbf{w}_0 = \mathbf{0}$ is the only case where the convention is exercised, and only at $i = 0$.

3.5.2 Convergence analysis and hypothesis verification for ELM LASSO

Theorem 3.1 (Convergence of SGPTL, adapted from [10, Slides 14–15], [7]). *Let $f : \mathbb{R}^H \rightarrow \mathbb{R}$ be convex with $f^* = \min_{\mathbf{w}} f(\mathbf{w}) > -\infty$ attained at some $\mathbf{w}^*$, and let $\{\mathbf{w}_i\}$ be the sequence generated by Algorithm 2. Assume the subgradients encountered along the run are uniformly bounded, $\|\mathbf{g}_i\|_2 \leq L$, and the stepsize-restricted rule $\beta_i \leq \gamma_i$ holds with $\gamma_i \geq \gamma_{\min} > 0$. Then the record values $\bar{f}_i = \min_{h \leq i} f(\mathbf{w}_h)$ converge to f^* , and once the target-level estimate has sharpened to f^* (i.e. $\delta_k \rightarrow 0$ and $f_{\text{ref}}^k \rightarrow f^*$, which Lemma 3.8 of [7] guarantees) the asymptotic rate, derived below from the per-step Polyak inequality for deflected directions [10, Slide 15], is*

$$\bar{f}_k - f^* \leq \frac{L \|\mathbf{w}_0 - \mathbf{w}^*\|_2}{\sqrt{\gamma_{\min} (k+1)}}, \quad (3.7)$$

so the worst-case complexity to attain $\bar{f}_k - f^* \leq \varepsilon$ is $\mathcal{O}(L^2/(\gamma_{\min} \varepsilon^2))$.

Proof. We verify the hypotheses of [7, Theorem 3.7] on Algorithm 2 and read off the conclusions. The framework of [7] is stated for σ_k -inexact subgradients, where $\sigma_k \geq 0$ bounds the oracle error; we compute $\mathbf{g}_i$ exactly via (3.6), so the inexactness parameter is zero. The remaining notation maps to ours via $\alpha_k \leftrightarrow \gamma_i$, $\beta_k \leftrightarrow \beta_i$, $\nu_k \leftrightarrow \alpha_i$, $\mu \leftrightarrow \rho$, $X = \mathbb{R}^H$ (no constraint), and $\lambda_k \leftrightarrow f(\mathbf{w}_i) - (f_{\text{ref}}^k - \delta_k)$.

(a) *Deflection-consistency condition (2.13).* The condition of [7, Lemma 2.4] requires $v_{k+1} = \tilde{d}_k$ whenever $d_k = \tilde{d}_k$, where $\tilde{d}_k$ is the unprojected deflected direction. In our unconstrained setting $X = \mathbb{R}^H$ no projection occurs, so $d_k = \tilde{d}_k$ at every iteration and Algorithm 2 sets $v_{k+1} = d_k$, so (2.13) holds trivially.

(b) *Stepsize condition (3.5) of [7].* Translated through the notation mapping above ($\alpha_k \leftrightarrow \gamma_i$, $\beta_k \leftrightarrow \beta_i$), the condition has two clauses:

- if $\lambda_k \geq 0$ then $\gamma_i \geq \beta_i \geq \beta^* > 0$ for some uniform constant $\beta^* > 0$ independent of i ;
- if $\lambda_k < 0$ then the step is null, $\alpha_i = 0$.

Case $\lambda_k > 0$ (target not yet reached). Line 12 of Algorithm 2 sets $\beta_i = \min(1, \gamma_i)$, and the clip (3.4) keeps $\gamma_i \in [\gamma_{\min}, 1]$. Hence $\beta_i \geq \gamma_{\min}$ for every i , and the uniform lower bound is satisfied with $\beta^* = \gamma_{\min}$. The other inequality $\gamma_i \geq \beta_i$ is just $\gamma_i \geq \min(1, \gamma_i)$, which holds by definition of min.

Case $\lambda_k \leq 0$ (target already attained at $\mathbf{w}_i$). The numerical safeguard of Section 3.4 skips the iteration with $\alpha_i = 0$, which is the second clause verbatim.

(c) *Target-level threshold condition (3.26) of [7].* The condition requires either $f_{\text{ref}}^\infty = f^* = -\infty$ or both $\liminf_{k \rightarrow \infty} \delta_k = 0$ and the series condition (3.25) of [7], $\sum_k \lambda_k / \|\mathbf{d}_k\|^2 = \infty$. Since on ELM LASSO $f^* > -\infty$, the first branch is excluded and we need the second. [7, Lemma 3.8] provides a concrete update strategy for $(f_{\text{ref}}^k, \delta_k)$ — the sufficient-descent test on $f_{k+1} \leq f_{\text{ref}}^k - \delta_k/2$, the target-infeasibility test on $r_k > R$, and the accumulate- r counter — that delivers both $\liminf_{k \rightarrow \infty} \delta_k = 0$ and (3.25). Lines 16–21 of Algorithm 2 implement that strategy verbatim under the identification $\mu \leftrightarrow \rho$, so condition (3.26) holds.

Subgradients uniformly bounded. The convex objective f of ELM LASSO is coercive and continuous; the iterates $\{\mathbf{w}_i\}$ remain in a compact sublevel set S_{c_0} for some $c_0 \geq f(\mathbf{w}_0)$, and ∂f is uniformly bounded on S_{c_0} by Lipschitz continuity of convex functions on compact sets [11, Slide 9].

Conditions (2.13), (3.5), (3.26) of [7] being verified with $\sigma^* = 0$, [7, Theorem 3.7] gives $f_{\text{ref}}^\infty \leq f^* + \sigma^* = f^*$, so $\bar{f}_i \rightarrow f^*$. To attach a rate to this convergence we work in the regime where the target-level update has driven $f_{\text{ref}}^k - \delta_k$ to f^* (Lemma 3.8 of [7]); the Polyak step at iteration i then uses target f^* exactly, $\alpha_i = \beta_i(f(\mathbf{w}_i) - f^*) / \|\mathbf{d}_i\|_2^2$.

Per-step descent. The stepsize-restricted Polyak inequality for a deflected direction [10, Slide 15] controls the squared distance to the optimum:

$$\|\mathbf{w}_{i+1} - \mathbf{w}^*\|_2^2 \leq \|\mathbf{w}_i - \mathbf{w}^*\|_2^2 - \beta_i(2 - \beta_i) \frac{(f(\mathbf{w}_i) - f^*)^2}{\|\mathbf{d}_i\|_2^2}. \quad (3.8)$$

Algorithm 2 sets $\beta_i = \min(1, \gamma_i)$ with $\gamma_i \in [\gamma_{\min}, 1]$, so $\beta_i = \gamma_i$ and $\beta_i(2 - \beta_i) = \gamma_i(2 - \gamma_i)$. On $[\gamma_{\min}, 1]$ the function $\gamma \mapsto \gamma(2 - \gamma)$ is increasing, so it is bounded below by $\gamma_{\min}(2 - \gamma_{\min}) \geq \gamma_{\min}$ (using $\gamma_{\min} \leq 1$).

Telescoping. Summing (3.8) from $i = 0$ to $i = k$ and dropping the non-negative left-hand side at index $k + 1$,

$$\gamma_{\min} \sum_{i=0}^k \frac{(f(\mathbf{w}_i) - f^*)^2}{\|\mathbf{d}_i\|_2^2} \leq \|\mathbf{w}_0 - \mathbf{w}^*\|_2^2.$$

Each $\mathbf{d}_i$ is a convex combination of subgradients along the run, so $\|\mathbf{d}_i\|_2 \leq \max_{h \leq i} \|\mathbf{g}_h\|_2 \leq L$; substituting,

$$\sum_{i=0}^k (f(\mathbf{w}_i) - f^*)^2 \leq \frac{L^2 \|\mathbf{w}_0 - \mathbf{w}^*\|_2^2}{\gamma_{\min}}.$$

The record value satisfies $\bar{f}_k - f^* = \min_{h \leq k} (f(\mathbf{w}_h) - f^*) \leq f(\mathbf{w}_i) - f^*$ for every $i \leq k$, so $(k + 1)(\bar{f}_k - f^*)^2$ lower-bounds the sum, giving (3.7).

Iteration count. Solving the right-hand side of (3.7) for ε yields

$$k + 1 \geq \frac{L^2 \|\mathbf{w}_0 - \mathbf{w}^*\|_2^2}{\gamma_{\min} \varepsilon^2}.$$

The unfloored Polyak bound (same derivation with $\gamma_{\min}$ replaced by 1) is $k + 1 \geq L^2 \|\mathbf{w}_0 - \mathbf{w}^*\|_2^2 / \varepsilon^2$, so the deflection floor inflates the iteration count by the factor $1/\gamma_{\min}$. With $\gamma_{\min} = 0.05$ this is a $20\times$ multiplier; the rate order $O(L^2/\varepsilon^2)$ is unchanged. $\square$

Verification of theorem assumptions for ELM LASSO. Convexity, coercivity, and uniqueness of $\mathbf{w}^*$ under full column rank of $\mathbf{X}$ follow from Proposition 1.1. The subgradient bound used in the proof requires only that f is Lipschitz on the iterate sublevel set, not globally: the smooth part $\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$ grows with $\|\mathbf{w}\|$, but $\{\mathbf{w}_i\}$ stays in a compact set ([7, Theorem 3.7]), so the local Lipschitz constant L in (3.7) is finite.

Complexity. The lower bound for first-order methods on nonsmooth convex problems is $\Omega(L^2/\varepsilon^2)$ [10, Slide 8], [4]; Algorithm 2 attains it. Deflection improves constants, not the order.

3.6 Expected practical behavior

Theorem 3.1 establishes that the record sequence $\bar{f}_i = \min_{h \leq i} f(\mathbf{w}_h)$ converges to f^* at the rate $\bar{f}_k - f^* \lesssim L \|\mathbf{w}_0 - \mathbf{w}^*\|_2 / \sqrt{\gamma_{\min}(k + 1)}$ of (3.7). From execution we therefore expect a concave curve of $\bar{f}_i - f^*$ on a semilog plot, with the residual gap closing as $1/\sqrt{i}$. The theorem bounds the record, not the raw

values $f(\mathbf{w}_i)$, so we expect $f(\mathbf{w}_i)$ to oscillate around $\bar{f}_i$ rather than decrease step by step.

The same bound depends on two ingredients which are problem-specific. The Lipschitz constant L scales with $\|\mathbf{X}\|_2$ and λ_{LASSO} (verification of theorem assumptions, Section 3.5.2); from (3.7) we therefore expect larger λ_{LASSO} to inflate L and shift the entire curve upward. The initial distance $\|\mathbf{w}_0 - \mathbf{w}^*\|_2$ enters multiplicatively, and this is what separates the two starts. The cold start $\mathbf{w}_0 = \mathbf{0}$ gives $\|\mathbf{w}_0 - \mathbf{w}^*\|_2 = \|\mathbf{w}^*\|_2$ exactly, so (3.7) predicts a higher curve; the OLS warm start gives $\|\mathbf{w}_{\text{OLS}} - \mathbf{w}^*\|_2$, which is small whenever OLS is already close to the LASSO minimiser — the two coincide at $\lambda_{\text{LASSO}} = 0$ and vary continuously with it — predicting a lower starting gap. How wide the resulting gap between the two starts is therefore depends on the instance: on a well-conditioned design where $\mathbf{w}_{\text{OLS}}$ sits near $\mathbf{w}^*$ the warm start should begin well below the cold one, whereas where $\mathbf{w}_{\text{OLS}}$ is far from $\mathbf{w}^*$ the two should be comparable. We run both starts on every dataset and report the respective analysis (Section 5.3).

The rate (3.7) controls $\bar{f}_i - f^*$, not the magnitude of individual coordinates of $\mathbf{w}_i$. A subgradient step $\mathbf{w}_{i+1} = \mathbf{w}_i - \alpha_i \mathbf{d}_i$ updates each coordinate continuously and never enforces $(\mathbf{w}_i)_j = 0$ exactly; combined with the rate (3.7), this leaves an inactive component at a magnitude no larger than the residual scale of the rate, $O(L/\sqrt{\gamma_{\min} i})$. We therefore do not expect SGPTL to produce hard sparsity at termination; recovery of the support will require a thresholding step set above this residual scale.

The optimality condition $0 \in \partial f(\mathbf{w}^*)$ of Section 1.4.3 is a set condition that holds at $\mathbf{w}^*$; it asks for some element of the set $\partial f(\mathbf{w}^*)$ to vanish, not for any element of $\partial f(\mathbf{w}_i)$ at a generic iterate. Under the sublinear rate (3.7) the iterates do not reach $\mathbf{w}^*$ within a finite budget, so we do not expect $\|\mathbf{g}_i\|_2 \rightarrow 0$ along the run regardless of which subdifferential representative is computed in (3.6). Termination will therefore rely on $i_{\max}$ or on a flat-window test on $\bar{f}_i$ rather than on a subgradient-based optimality check.

Chapter 4

Algorithm Comparison

In this chapter, we will compare the two algorithms together to analyze their properties and trade-offs.

4.1 Core strategy: how each algorithm handles non-smoothness

IRLS: smoothing via majorization

IRLS replaces f with a smooth quadratic majorant $Q(\mathbf{w}, \mathbf{w}_k)$ (2.2), reducing each step to a weighted L_2 problem solved in closed form via the normal equations (2.3).

DSM: direct subgradient with memory

DSM descends along a subgradient $\mathbf{g} \in \partial f(\mathbf{w}_i)$ (3.5.1), deflected with the previous direction (3.1.1) with Polyak step using a self-adapting target estimate of f^* (3.3).

4.2 Convergence behaviour

4.2.1 Monotonicity

The majorization-minimization construction of Section 2.2 guarantees IRLS is **monotone**: $f(\mathbf{w}_{k+1}) \leq f(\mathbf{w}_k)$. DSM is **non-monotone**: only the record $\bar{f}^i = \min_{h \leq i} f(\mathbf{w}_h)$ is monotone, so DSM returns $\mathbf{w}_{\text{best}}$ and stops on a fixed budget.

4.2.2 Rate

IRLS achieves a **linear** rate (2.5), DSM achieves the optimal **sublinear** $O(L^2/\varepsilon^2)$ rate for nonsmooth convex functions (Theorem 3.1). Let's analyze the behaviour when we lower ε :

- DSM: $\bar{f}_k - f^* \lesssim L \|\mathbf{w}_0 - \mathbf{w}^*\|_2 / \sqrt{\gamma_{\min} k}$ gives $k \propto 1/(\gamma_{\min} \varepsilon^2)$. Lowering ε by a factor 10 therefore multiplies the iteration count by $10^2 = 100$.
- IRLS: $f(\mathbf{w}_{k+1}) - f^* \leq r(f(\mathbf{w}_k) - f^*)$ with $r \in (0, 1)$ (Theorem 2.2) gives $k \propto \log(1/\varepsilon)/\log(1/r)$. Lowering ε by a factor 10 *adds* the constant $\log 10/\log(1/r)$ iterations, independent of ε .

For more precision of the target solution, DSM requires a polynomial number of additional steps, meanwhile IRLS requires only logarithmic (additively constant per decade).

4.2.3 Convergence on the ELM problem

As expressed before, $\mathbf{X}$ has full column rank, so f is μ -strongly convex with $\mu = \lambda_{\min}(\mathbf{X}^\top \mathbf{X}) > 0$ and admits a unique minimizer $\mathbf{w}^*$ (1.1). Both algorithms are theoretically guaranteed to converge on this problem, but with different notions of convergence: Theorem 2.2 gives IRLS iterates $\mathbf{w}_k \rightarrow \mathbf{w}^*$ at a linear rate, while Theorem 3.1 gives SGPTL only the weaker guarantee $\bar{f}_i \rightarrow f^*$ on the record sequence $\bar{f}_i = \min_{h \leq i} f(\mathbf{w}_h)$, but single iterates $\{\mathbf{w}_i\}$ may oscillate.

4.3 Computational cost

4.3.1 Per-iteration cost

Operation	IRLS	DSM
Weight / direction update	$O(H)$	$O(H)$
Matrix-vector product(s)	—	$O(MH)$
Linear system solve	$O(H^3)$ (Cholesky) or $O(pH^2)$ (CG)	—
Total per iteration	$O(H^3)$	$O(MH)$

Table 4.1: Per-iteration computational cost of IRLS and DSM.

IRLS solves an $H \times H$ SPD system per iteration (2.3): $O(H^3/3)$ via Cholesky, or $O(pH^2)$ via p CG steps when H is large and $\mathbf{Q}_k$ well-conditioned.

DSM uses two matrix-vector products per iteration ($\mathbf{X}\mathbf{w}_i$ and $\mathbf{X}^T(\cdot)$, each $O(MH)$) (3.5.1).

4.3.2 Total cost

Accounting for both the precomputation phase and the iteration loop, the total costs are

$$\begin{aligned}
\text{IRLS (OLS warm start): } & O(MH^2) + O(MH) + O(H^3) + k_{\text{IRLS}} \cdot O(H^3) \\
\text{IRLS (cold start): } & O(MH^2) + O(MH) + k_{\text{IRLS}} \cdot O(H^3) \\
\text{SGPTL with OLS warm start: } & O(MH^2) + O(H^3) + k_{\text{DSM}} \cdot O(MH), \\
\text{SGPTL with cold start } \mathbf{w}_0 = \mathbf{0}: & k_{\text{DSM}} \cdot O(MH).
\end{aligned}$$

On the ELM problem, $M \gg H$ is typical, so $O(MH^2)$ dominates IRLS's total. IRLS therefore wins whenever $k_{\text{DSM}} \gtrsim H$: the warm-start variant of SGPTL already pays the same $O(MH^2)$ overhead and adds the iteration cost on top, while the cold-start variant trades the overhead for k_{DSM} matrix-vector products at $O(MH)$ each. Since Theorem 3.1 forces $k_{\text{DSM}} = O(L^2/\varepsilon^2)$ with L growing in H (the ℓ_1 subgradient has norm $\leq \lambda_{\text{LASSO}}\sqrt{H}$), the inequality $k_{\text{DSM}} > H$ holds at every H we test for any reasonable accuracy.

4.4 Solution quality

4.4.1 Sparsity

IRLS yields numerically sparse iterates whenever we have small components of $\mathbf{w}$, which are sent towards ε (theoretically 0) (2.7). DSM gives only **approximate sparsity**: an explicit thresholding step is needed (3.6).

4.4.2 Accuracy

At equal budget, IRLS reaches much higher accuracy on small-to-moderate problems thanks to its linear rate. DSM is competitive only when $H^2 \gg M$ (Table 4.1).

4.5 Comparison summary for the ELM problem

Making the assumption that $M \gg H$, IRLS algorithm is preferred: the $O(MH^2)$ precomputation of $\mathbf{A}$ amortizes over a few cheap Cholesky solves, inducing sparsity, while only having to tune ε_{thr} . DSM is the better choice only when $H^2 \gg M$ or $\mathbf{X}^T\mathbf{X}$ does not fit in memory, and the accuracy target is moderate ($\varepsilon \sim 10^{-3}$).

Criterion	IRLS	DSM
Approach	Smoothing (MM)	Direct subgradient
Monotone	Yes	No (record value only)
Convergence rate	Linear	Sublinear $O(L^2/\varepsilon^2)$
Per-iteration cost	$O(H^3)$	$O(MH)$
Precomputation	$O(MH^2) + O(MH) + O(H^3)$	None
Exact sparsity	Yes	No (needs thresholding)
Parameters to tune	ε_{thr}	δ_0, ρ, R, β
Stopping criterion	Reliable (iterate change)	Unreliable (fixed budget)
Preferred when	High accuracy, $H \ll M$	Large H , moderate accuracy

Table 4.2: Summary comparison of IRLS and DSM on the ELM LASSO problem.

Chapter 5

Experimental Results

This chapter tests the IRLS and SGPTL implementations against the expected results stated in Chapters 2 and 3 and against the algorithmic comparison of Chapter 4. The guiding questions are the two of a numerical study: do the algorithms behave on ELM LASSO as the theory of Chapters 2 and 3 leads us to expect, and how do their cost–accuracy trade-offs play out against the comparison drawn in Chapter 4.

5.1 Experimental setup and datasets

Implementation. The code under `progetto/code/src/` is split into five modules: `lasso_utils.py` (objective, smooth-part gradient, subgradient selection), `linear_solvers.py` (Cholesky via `scipy.linalg.cho_factor/cho_solve` and a hand-rolled Jacobi-preconditioned CG), `irls.py` (Algorithm 1), `deflected_subgradient.py` (Algorithm 2), and `elm.py` (random hidden layer). Core algorithms are written from scratch (reusing only NumPy/SciPy as primitives). A pytest suite cross-checks both algorithms against `sklearn.linear_model.Lasso` and verifies the surrogate-descent and record-monotonicity invariants iterate by iterate.

Reference solution. We obtain $\mathbf{w}^*$ from `sklearn.linear_model.Lasso` (coordinate descent, `tol=10-12`, `105` iterations, `fit_intercept=False`). Sklearn’s loss has a $1/(2M)$ prefactor on the squared residual and weights the ℓ_1 term by α , so the equivalent of (1.3) is obtained by passing $\alpha_{\text{sklearn}} = \lambda_{\text{LASSO}}/M$; we then evaluate $f^* = f(\mathbf{w}^*)$ from (1.3) directly. The returned $\mathbf{w}^*$ satisfies the optimality condition (1.5) below 10^{-3} on every instance. On the ELM-transformed real data, where coordinate descent does not converge in a realistic budget, we replace this reference with the IRLS-converged value cross-validated against CVXPY’s interior-point solver Clarabel (Section 5.6).

5.1.1 Datasets

We use three instances spanning two conditioning regimes.

Synthetic LASSO benchmark (`src/data_generation.py`). We sample a sparse $\mathbf{w}_{\text{true}} \in \mathbb{R}^H$ (10–15% non-zero, entries $\sim \mathcal{N}(0, 1)$), draw $\mathbf{X} \in \mathbb{R}^{M \times H}$ with i.i.d. Gaussian columns normalised to unit ℓ_2 norm, and set $\mathbf{y} = \mathbf{X}\mathbf{w}_{\text{true}} + \boldsymbol{\varepsilon}$, $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, 0.05^2 \mathbf{I})$. This is the benign regime for LASSO. We pick $M \geq 2H$ to keep $\kappa(\mathbf{X}^\top \mathbf{X})$ at a moderate constant by random-matrix concentration [14], the planted $\mathbf{w}_{\text{true}}$ provides the ground truth for support-recovery metrics, and the noise std 0.05 puts the SNR in the moderate-noise regime (about 5–10% of the signal magnitude). It validates the algorithms in a well-conditioned setting but does *not* reproduce the conditioning of ELM data.

ELM-transformed real datasets (`diabetes`, `california`). We load `diabetes` ($n = 442$, $d = 10$) [9] and `california_housing` ($n = 20640$, $d = 8$) [17] from scikit-learn, split 80/20, standardise features and target on the train split, and apply the ELM map $\mathbf{X}_{\text{hidden}} = \sigma(\mathbf{X} \mathbf{W}_1^\top)$ with H sigmoid units and a fixed random $\mathbf{W}_1$. We use $H = 200$ for the main experiments and $H = 50$ where an independent solver (`cvxpy.CLARABEL`) is affordable for cross-validation. The sigmoid breaks the unit-norm column property and inflates $\kappa(\mathbf{X}_{\text{hidden}}^\top \mathbf{X}_{\text{hidden}})$ to $\sim 10^6$ on both datasets — orders of magnitude worse than the synthetic design, which is the stress regime we want to probe.

Limitations. (i) The synthetic benchmark lives in the well-conditioned, incoherent regime by construction, so positive results there do not extrapolate to ill-conditioned designs. (ii) The real datasets have no ground-truth support, so the *support-recovery* metrics of Section 5.5.1 (precision/recall/ F_1 against the planted support) apply only to the synthetic instance; sparsity counts at a fixed threshold remain well-defined and are reported on real data alongside training-objective gaps and test MSE. (iii) On `california` at $\lambda_{\text{LASSO}} = 0.1$ the OLS solution already attains f^* to within $5 \cdot 10^{-1}$ (the ℓ_1 term is weak relative to the data scale), so a warm-started SGPTL has essentially nothing to optimise; we account for this explicitly in Sections 5.3 and 5.6.

5.2 Convergence and agreement with theory

Setting. Synthetic convergence instance, $H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, sparsity 10%. Both algorithms are run from the same OLS warm start $\mathbf{w}_0 = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ so that any difference in the curves is attributable to the algorithm and not to the initialisation; the one-time Cholesky cost of forming $\mathbf{w}_0$ (Section 3.4.1) is charged to both wall-clock totals. The relevant objective values of this instance are $f(\mathbf{0}) \approx 76$ at the origin, $f(\mathbf{w}_0) \approx 1.56$ at the warm start, and $f^* \approx 1.13$ at the sklearn reference, which puts the initial gap $f(\mathbf{w}_0) - f^* \approx 0.43$ — the y -intercept of Figure 5.1.

IRLS: linear rate (Theorem 2.2). The expected behaviour from Chapter 2 is linear convergence of the MM iteration. Figure 5.1 (left) confirms it: after

a fast transient the gap decreases at a constant per-iteration factor ≈ 0.848 in the asymptotic region, reaching $1.5 \cdot 10^{-6}$ in 100 iterations (iterate change $5.6 \cdot 10^{-6}$). The late flattening toward $\sim 10^{-6}$ is the accuracy floor of the smoothed surrogate at $\varepsilon_{\text{thr}} = 10^{-8}$ (Section 5.4), not a breakdown of the rate.

SGPTL: sublinear rate (Theorem 3.1). The expected behaviour from Chapter 3 is an $O(1/\sqrt{i})$ record gap and a non-monotone current value. Figure 5.1 (right, 8000 iterations, $\beta = 1$, $\gamma_{\min} = 0.05$, $\delta_0 = 0.1 f(\mathbf{w}_0)$, $\rho = 0.7$) confirms both: the record $\bar{f}^i - f^*$ follows the sublinear envelope down to $4.8 \cdot 10^{-5}$, and each decade of accuracy costs a $\sim 100\times$ increase in iterations, exactly the $\varepsilon \mapsto \varepsilon/10 \Rightarrow i \mapsto 100i$ scaling of the $O(L^2/\varepsilon^2)$ bound. The current value oscillates around the record, as the theory allows (the bound controls the record, not the raw iterate).

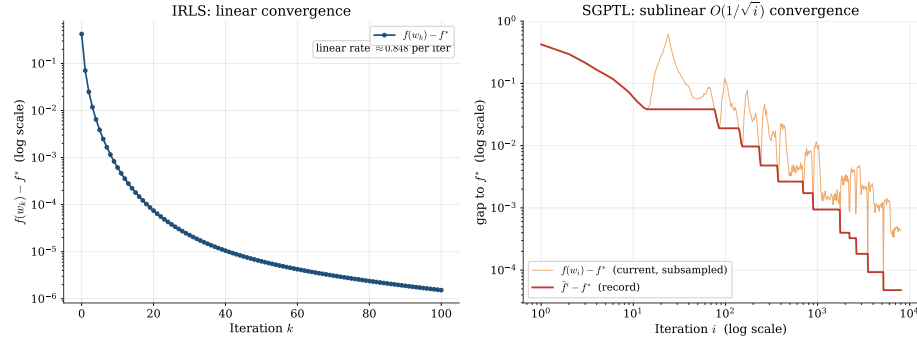


Figure 5.1: Optimality gap versus iteration on $H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$. Left: IRLS, semilog axis; the annotation is the empirical per-iteration factor in the linear region. Right: SGPTL, log-log axes, current value (orange, subsampled log-uniformly) over the record (red). The record traces the $O(1/\sqrt{i})$ envelope of Theorem 3.1.

Against wall-clock time (Figure 5.2) SGPTL’s cheaper iteration ($O(MH)$ vs $O(H^3)$) does not offset the slower rate at this size: IRLS reaches 10^{-6} in ~ 3 ms, SGPTL drops below 10^{-2} within ~ 10 ms and then crawls to $\sim 5 \cdot 10^{-5}$ over ~ 150 ms.

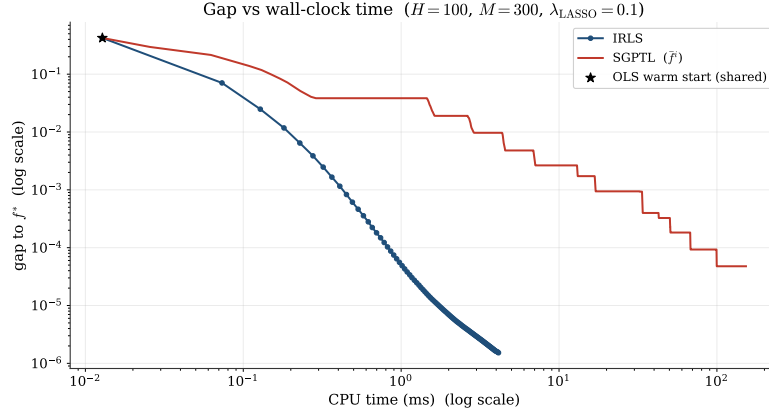


Figure 5.2: Optimality gap versus CPU time on the instance of Figure 5.1, log-log axes.

Both convergence curves above track the SGPTL *record* $\bar{f}^i = \min_{h \leq i} f(\mathbf{w}_h)$ rather than the raw current value $f(\mathbf{w}_i)$, because — as expected from Section 3.3 — the subgradient dynamics are not monotone, and only the record carries the rate guarantee of Theorem 3.1. Figure 5.3 makes this explicit: the current gap spikes well above the record at several points along the run, while the record is monotone by construction. This is also the practical reason the algorithm tracks and returns $\mathbf{w}_{\text{best}}$ rather than the last iterate.

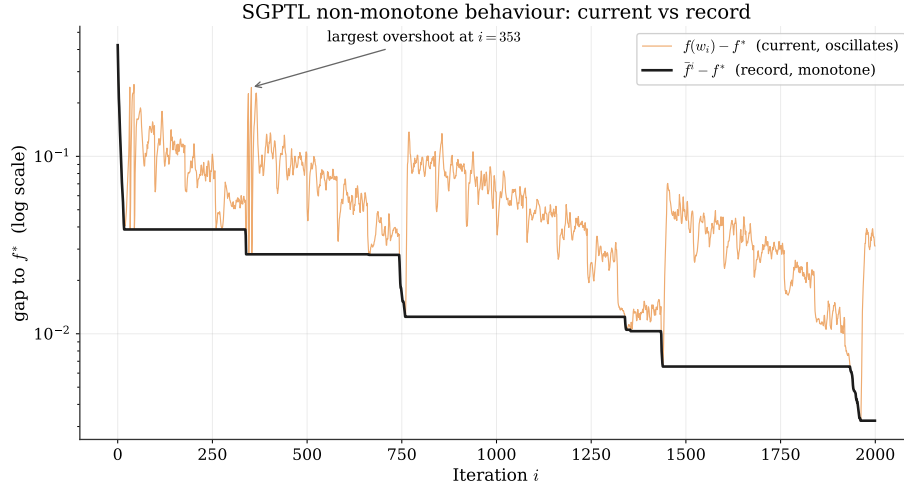


Figure 5.3: SGPTL on the convergence instance, first 2000 iterations, semilog y . The current gap (orange) overshoots the record (black) repeatedly; the largest overshoot is annotated.

5.3 Cold versus warm start

Setting. We run *both* algorithms from *both* initialisations on all three instances: the synthetic benchmark ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$) and the two ELM-transformed real datasets ($H = 200$). Both Theorem 2.2 (IRLS) and Theorem 3.1 (SGPTL) admit any $\mathbf{w}_0$; what changes between the starts is the initial distance $\|\mathbf{w}_0 - \mathbf{w}^*\|$, which enters their convergence bounds multiplicatively. Table 5.1 collects the final gaps; the rest of the section interprets them.

algorithm	start	synthetic	diabetes	california
IRLS	warm	$1.8 \cdot 10^{-8}$ (940)	$<10^{-11}$ (894)	$<10^{-11}$ (102)
IRLS	cold	$1.8 \cdot 10^{-8}$ (922)	$<10^{-11}$ (1 118)	$<10^{-11}$ (115)
SGPTL	warm	$4.8 \cdot 10^{-5}$	2.62	0.46
SGPTL	cold	$2.4 \cdot 10^{-5}$	1.74	~ 64

Table 5.1: Final gap $|f(\mathbf{w}_{\text{final}}) - f^*|$ on each instance; the parenthesised number after each IRLS entry is the iteration count at which $\varepsilon_{\text{stop}} = 10^{-12}$ triggers. IRLS reaches f^* to working precision on all three instances from both starts (limited by the $\varepsilon_{\text{thr}} = 10^{-8}$ smoothing floor on the synthetic, by $\varepsilon_{\text{stop}}$ on the real data); the warm vs cold contrast is therefore in iteration count, not final accuracy. SGPTL runs 8 000 iterations everywhere with the defaults of Section 5.4; here the final gap is the differentiator. The f^* reference is the sklearn-converged value on the synthetic and the IRLS+CVXPY-Clarabel value on the real data (Section 5.6).

Order-of-magnitude check on the SGPTL real-data gaps. The SGPTL gaps on the real datasets (1.74 on *diabetes*, ~ 64 on *california*) are the order of magnitude that the $O(1/\sqrt{k})$ rate of Theorem 3.1 predicts. The theorem reads $\bar{f}^k - f^* \leq L \|\mathbf{w}_0 - \mathbf{w}^*\| / \sqrt{\gamma_{\min}(k+1)}$; collapsing the numerator $L \|\mathbf{w}_0 - \mathbf{w}^*\|$ into the initial objective gap $g_0 = f(\mathbf{w}_0) - f^*$ (same scaling) gives the rule of thumb $\bar{f}^k - f^* \sim g_0/\sqrt{k}$. With cold start and standardised targets, $g_0 \approx M_{\text{train}}/2$, so at $k = 8\,000$ the heuristic predicts ~ 0.28 on the synthetic, ~ 1.4 on *diabetes*, ~ 66 on *california*: the real-data numbers match to within a factor of two (1.74 and ~ 64). On the synthetic the observed gap $2.4 \cdot 10^{-5}$ sits five orders of magnitude below this heuristic, because the worst-case L and sublevel-set radius are loose on a well-conditioned design with normalised columns and the bound is not tight in that regime. The same looseness shows on the real data once k grows (Section 5.3.1), driven by the δ -contraction staircase focusing descent in bursts and by the local landscape near $\mathbf{w}^*$ being essentially quadratic on the identified support. Theorem 3.1 still bounds the gap on these runs; the empirical margin to the envelope widens as k grows.

IRLS. Both starts converge to f^* on every instance, as the linear rate of Theorem 2.2 allows — the IRLS curve on each panel of Figure 5.4 flattens to the

$\varepsilon_{\text{stop}}/\varepsilon_{\text{thr}}$ floor regardless of start. The differentiator is therefore the *iteration count*: cold needs $\sim 25\%$ more iterations than warm on **diabetes** (1118 vs 894) and $\sim 13\%$ more on **california** (115 vs 102), while on the synthetic the two are within rounding. The pattern matches the bound: the factor $\|\mathbf{w}_0 - \mathbf{w}^*\|$ enters multiplicatively, so a closer start saves a small but consistent number of iterations on each instance. The warm start is the natural IRLS default and we use it elsewhere in this report.

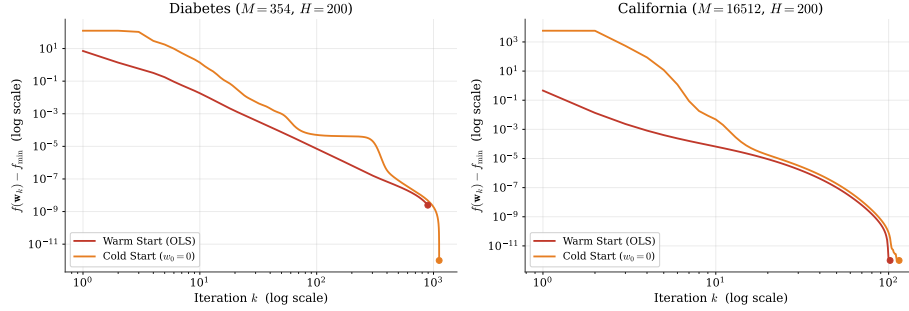


Figure 5.4: IRLS warm (OLS) vs cold ($\mathbf{w}_0 = \mathbf{0}$) start on the two ELM-transformed real datasets ($H = 200$, $\lambda_{\text{LASSO}} = 0.1$, $k_{\text{max}} = 2000$, $\varepsilon_{\text{stop}} = 10^{-12}$), log-log axes. Both starts converge to f^* on each instance, as the linear rate of Theorem 2.2 allows; the warm curve sits below the cold one throughout because $\|\mathbf{w}_0 - \mathbf{w}^*\|$ enters the bound multiplicatively. The synthetic analogue is in Appendix B.

SGPTL. The sublinear rate makes the warm-cold contrast much less decisive than for IRLS: the bound (3.7) shrinks the residual as $1/\sqrt{i}$, so unless $\mathbf{w}_{\text{OLS}}$ is already very close to $\mathbf{w}^*$ the two starts end the budget at comparable gaps. On the synthetic instance the two ends within a factor of two (Figure 5.5: cold $2.4 \cdot 10^{-5}$ with 26 δ -contractions, warm $4.8 \cdot 10^{-5}$ with 22) — consistent with the well-conditioned regime where the warm head start is amortised quickly. On **diabetes**-ELM the cold trajectory dips below the warm one past $\sim 10^2$ iterations and stays there (1.74 vs 2.62; contraction counts 23 vs 25, so the difference is not driven by the patience schedule). On **california**-ELM the picture is different: $f(\mathbf{w}_{\text{OLS}})$ already sits within 0.46 of f^* before SGPTL takes its first step, and the Polyak stepsize $\alpha_i = \beta_i(f(\mathbf{w}_i) - \text{target})/\|\mathbf{d}_i\|_2^2$ shrinks with the current gap, so the iterate barely moves over 8000 iterations. The warm-start number 0.46 therefore measures how close OLS already is to f^* on this instance, not the residual that SGPTL has reduced from it; correspondingly, the cold start, which begins ~ 5900 away from f^* , closes two decades of gap (down to ~ 64) over the same budget. The contrast is plotted on the real-data convergence figure (Figure 5.12, Section 5.6, which overlays both SGPTL starts).

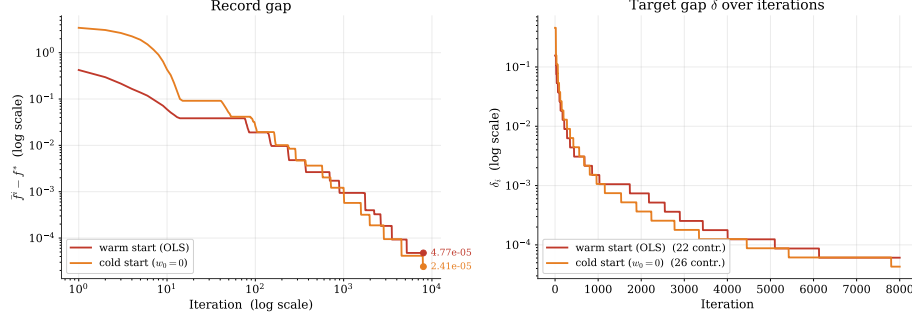


Figure 5.5: SGPTL warm vs cold start on the synthetic instance ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, $i_{\max} = 8000$, $\rho = 0.7$, $\gamma_{\min} = 0.05$, $R = 1$). Left: record gap, log-log; both descend sublinearly. Right: δ_i staircase (26 contractions cold, 22 warm).

Defaults. For IRLS we use the OLS warm start everywhere in this report — it is the natural pick given the linear-rate amplification. For SGPTL we adopt the cold start $\mathbf{w}_0 = \mathbf{0}$ as the default (it is the method’s natural initialisation and the configuration in which it actually optimises across instances) and pass an explicit warm start only where its final gap is empirically smaller, as on `california` in Table 5.8.

5.3.1 SGPTL beyond the submitted iteration budget

The estimate above predicts the SGPTL real-data gaps at $k = 8000$ from the rule of thumb $g_0/\sqrt{k}$. We push the experiment much further to see how the gap evolves: SGPTL cold to $i_{\max} = 10^7$ on `diabetes-ELM` (3.3 min wall-clock) and to $i_{\max} = 10^6$ on `california-ELM` (23 min). Figure 5.6 overlays the empirical record gap against the heuristic envelope $g_0/\sqrt{i}$.

The empirical trace stays *below* the envelope and descends faster than its worst-case slope (Table 5.2). On `diabetes` the gap drops from 0.26 at $k = 8000$ to $\leq 10^{-12}$ by $k = 8 \cdot 10^6$ — SGPTL reaches the floating-point floor on this instance with enough budget. On `california` it descends from 41 at $k = 8000$ to $1.2 \cdot 10^{-6}$ at $k = 8 \cdot 10^5$.

	diabetes-ELM ($g_0 = 124$)		california-ELM ($g_0 = 5898$)	
k	observed	$g_0/\sqrt{k}$	observed	$g_0/\sqrt{k}$
8 000	$2.60 \cdot 10^{-1}$	1.39	$4.14 \cdot 10^1$	$6.59 \cdot 10^1$
80 000	$2.27 \cdot 10^{-5}$	$4.39 \cdot 10^{-1}$	$7.50 \cdot 10^{-1}$	$2.09 \cdot 10^1$
800 000	$3.40 \cdot 10^{-7}$	$1.39 \cdot 10^{-1}$	$1.20 \cdot 10^{-6}$	6.59
8 000 000	$\leq 10^{-12}$	$4.39 \cdot 10^{-2}$	—	—
10 000 000	$\leq 10^{-12}$	$3.92 \cdot 10^{-2}$	—	—

Table 5.2: Observed record gap of SGPTL versus the $g_0/\sqrt{k}$ envelope, cold start on the ELM-transformed real datasets. The empirical descent is faster than the worst-case envelope at every k ; on **diabetes** SGPTL reaches the floating-point floor at $k \approx 8 \cdot 10^6$.

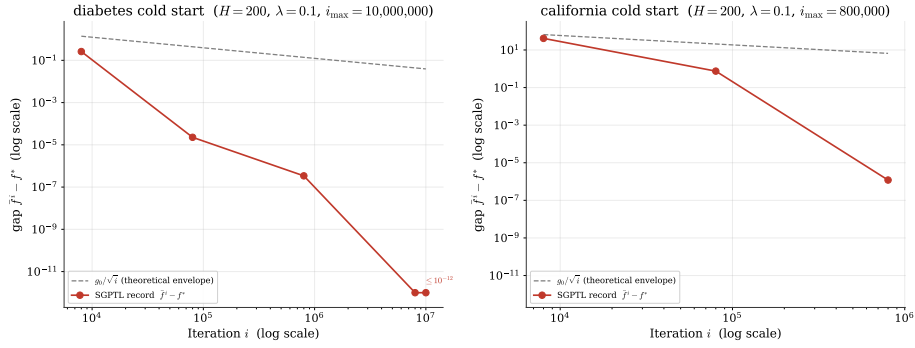


Figure 5.6: SGPTL at large iteration budgets, cold start on the ELM-transformed real datasets ($H = 200$, $\lambda_{\text{LASSO}} = 0.1$), log-log. Empirical record gap (solid) vs the heuristic envelope $g_0/\sqrt{i}$ (dashed grey). Left: **diabetes** to $i_{\max} = 10^7$, reaches the 10^{-12} floor by $\sim 8 \cdot 10^6$ iterations. Right: **california** to $i_{\max} = 8 \cdot 10^5$, gap $\sim 10^{-6}$ at the budget.

5.4 Hyperparameter tuning and technique choices

This section reports sensitivity sweeps on each algorithm separately: the IRLS parameters ε_{thr} and λ_{LASSO} and the choice of inner solver (Cholesky vs Jacobi-preconditioned CG) below, the SGPTL parameters $\gamma_{\min}$, δ_0 and ρ after.

5.4.1 IRLS: ε_{thr} and λ_{LASSO}

We sweep both at $H = 100$, $M = 400$, 100 iterations (Figure 5.7). For $\varepsilon_{\text{thr}} \in \{10^{-4}, \dots, 10^{-12}\}$ the final gap tracks ε_{thr} ($1.4 \cdot 10^{-6}$, $1.5 \cdot 10^{-8}$, $5.3 \cdot 10^{-10}$ at $10^{-6}, 10^{-8}, 10^{-10}$); 10^{-4} is too loose (no sparsity) and 10^{-12} hits floating-point limits, so the 10^{-8} – 10^{-10} default is the sweet spot. Sweeping $\lambda_{\text{LASSO}} \in \{0.01, \dots, 1.0\}$ leaves the convergence speed roughly invariant (λ enters $\mathbf{Q}_k$ as a

bounded conditioning factor) while sparsity moves monotonically from 11% to 95%, as the optimality condition (1.5) implies.

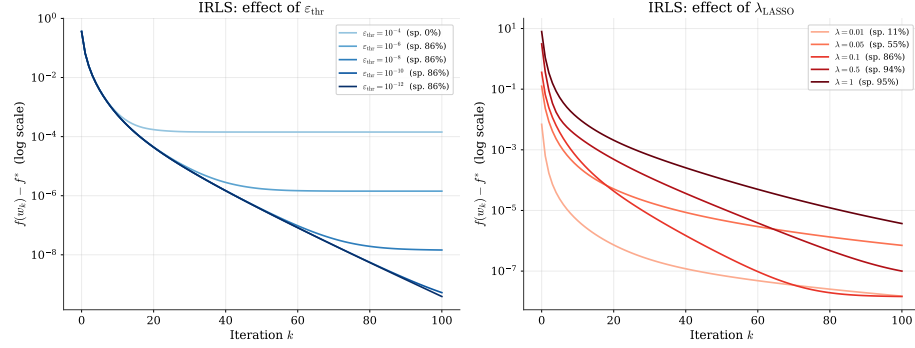


Figure 5.7: IRLS sensitivity. Left: ε_{thr} at $\lambda_{\text{LASSO}} = 0.1$. Right: λ_{LASSO} at $\varepsilon_{\text{thr}} = 10^{-8}$.

5.4.2 IRLS: inner solver (Cholesky vs. CG)

Chapter 2 leaves the inner solver of $\mathbf{Q}_k \mathbf{w}_{k+1} = \mathbf{b}$ open between Cholesky ($O(H^3/3)$) and CG ($O(pH^2)$ per step). Table 5.3 compares them on $H = 100$, $M = 400$: both reach the stopping criterion at iteration 265 with identical trajectories, but exact Cholesky takes 11.7 ms against 24.4 ms for Jacobi-preconditioned CG. As IRLS drives inactive weights to the cap $1/\varepsilon_{\text{thr}} = 10^8$, $\kappa(\mathbf{Q}_k)$ blows up; the Jacobi preconditioner on $\text{diag}(\mathbf{Q}_k)$ is what keeps CG matching Cholesky iteration-by-iteration, but the iterative overhead still doubles the wall-clock time. In the project regime ($M \gg H$, $H \leq 1000$) Cholesky is the default — faster, MM-descent guaranteed unconditionally, no inner tolerance to tune; CG remains a drop-in for H large enough that the $O(H^3)$ term dominates.

Solver	Total time (ms)	Iterations	Sparsity at 10^{-6}	Converged
Cholesky	11.7	265	86%	Yes
CG	24.4	265	86%	Yes

Table 5.3: IRLS solver comparison on $H = 100$, $M = 400$, $\lambda_{\text{LASSO}} = 0.1$, $\varepsilon_{\text{thr}} = 10^{-8}$, $k_{\text{max}} = 300$, OLS warm start. Both reach $\varepsilon_{\text{stop}} = 10^{-10}$ at iteration 265; sparsity at the final iterate under $|w_i| < 10^{-6}$.

5.4.3 SGPTL: the deflection floor $\gamma_{\min}$

Theorem 3.1 requires $\gamma_i \geq \gamma_{\min} > 0$ to keep the stepsize clip $\beta_i = \min(\beta, \gamma_i)$ uniformly bounded away from zero (Section 3.2); the greedy formula (3.4) of Section 3.2.1 can otherwise drive γ_i arbitrarily close to zero near stationarity.

We run two sweeps below to (i) check whether the floor is needed in practice on ELM LASSO and (ii) calibrate its value.

Does the floor matter? (binary test, $\gamma_{\min} \in \{0, 0.05\}$). We compare the algorithm with no floor ($\gamma_{\min} = 0$, the raw greedy) against the floored variant ($\gamma_{\min} = 0.05$) on a 3×5 grid: three sizes (H, M) equal to $(50, 150)$, $(100, 300)$ and $(200, 600)$, five seeds each. The same pattern emerges across the 30 runs (Figure 5.8; aggregate over the medium-size block):

$\gamma_{\min}$	final gap	$\Pr\{\gamma_i \leq 0.01\}$	$\Pr\{\text{skip fires}\}$	$\delta\text{-contr.}$
0.00	$4.4 \cdot 10^{-2}$	99%	98%	~ 2
0.05	$1.2 \cdot 10^{-4}$	0%	0%	~ 19

The mechanism behind the failure mode of $\gamma_{\min} = 0$ is a chain reaction visible on the figure’s leftmost panel. Near stationarity the greedy γ^* from (3.4) approaches zero because consecutive subgradients align and the numerator $\|\mathbf{d}_{i-1}\|^2 - \langle \mathbf{g}_i, \mathbf{d}_{i-1} \rangle$ vanishes. With no floor, γ_i collapses below 10^{-10} from roughly iteration 50 onward and stays there for the rest of the run. The clip $\beta_i = \min(1, \gamma_i)$ collapses with it, the Polyak step (3.2) becomes positive but minuscule, and once f_i no longer beats the target the numerator-skip safeguard fires on 98% of iterations: the iterate freezes and δ contracts only ~ 2 times instead of ~ 19 , leaving the gap stuck at $\sim 4 \cdot 10^{-2}$ (centre and right panels). With $\gamma_{\min} = 0.05$ the floor binds whenever the greedy would hit zero, β_i stays ≥ 0.05 , the Polyak step remains effective, and the staircase descent of δ that the theory predicts takes place — the gap drops by two more orders of magnitude. The chain reaction is structural to ELM LASSO with the greedy γ (the failure depends on the algorithm’s choice of γ , not on the specific instance), and we observed it on all 15 unfloored seeds.

Which value of $\gamma_{\min}$ (calibration sweep). We sweep $\gamma_{\min}$ over 0.05, 0.1, 0.2 and 0.5 on the convergence instance. The final gap lies in $[10^{-7}, 10^{-5}]$ across the four values, with $\gamma_{\min} = 0.05$ marginally best. Larger values force the clip to bind on iterations where the greedy γ_i would otherwise lie strictly inside $(\gamma_{\min}, 1]$, so we adopt $\gamma_{\min} = 0.05$.

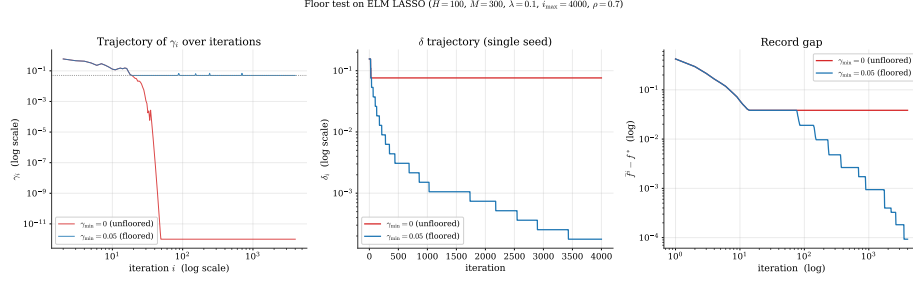


Figure 5.8: Floor test on ELM LASSO ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, $i_{\max} = 4000$, $\rho = 0.7$), single seed shown. Left: trajectory of γ_i over the run (log-log). Without the floor, γ_i collapses below 10^{-10} around iteration 50 and stays there; with $\gamma_{\min} = 0.05$ it sits on the floor (dotted grey line) for the rest of the run. Centre: target gap δ_i ; the unfloored δ contracts twice and stops, the floored one descends in the staircase the theory expects. Right: record gap $\bar{f}^i - f^*$; the floor buys two orders of magnitude. The aggregate over the 30 runs of the binary grid agrees with this single-seed snapshot.

5.4.4 SGPTL: δ_0 and ρ

Of the four target-level parameters of SGPTL, β and R are settled analytically in Section 3.4.1 and not swept here. The remaining two, $\delta_0 = c f(\mathbf{w}_0)$ and ρ , have intrinsically empirical tradeoffs and are calibrated below. The sweeps of Figure 5.9 ($H = 100$, $M = 400$, 5000 iterations) show the method is robust to both: $c \in \{0.01, \dots, 1.0\}$ at $\rho = 0.7$ gives final gaps in $[5.6 \cdot 10^{-5}, 1.6 \cdot 10^{-4}]$, within one order of magnitude across two decades of δ_0 ; $\rho \in \{0.3, \dots, 0.95\}$ at $c = 0.1$ gives $1.98 \cdot 10^{-5}$ up to $8.12 \cdot 10^{-4}$ with 7 to 91 contractions. Small ρ wins on this instance, but a smoke test over the synthetic and the two ELM matrices places the per-instance best at $\rho = 0.3, 0.3, 0.7$; only $\rho \in [0.3, 0.7]$ is within a factor of two of the best everywhere, and $\rho \geq 0.9$ loses by one to two orders of magnitude. We adopt $\rho = 0.7$ and $c = 0.1$ as robust defaults.

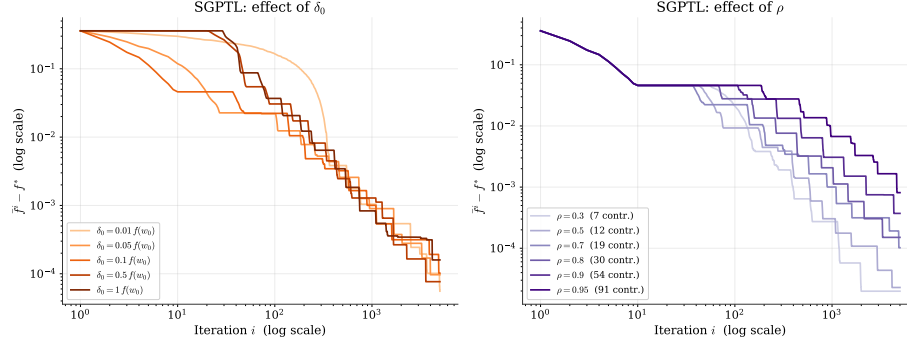


Figure 5.9: SGPTL sensitivity ($H = 100$, $M = 400$, OLS warm start, 5000 iterations). Left: δ_0 as a fraction of $f(\mathbf{w}_0)$ at $\rho = 0.7$. Right: ρ at $\delta_0 = 0.1 f(\mathbf{w}_0)$; gaps and contraction counts grow monotonically with ρ .

5.4.5 SGPTL: δ_0 against the f^* -based scale

The choice $\delta_0 = c f(\mathbf{w}_0)$ is settled on principled grounds in Section 3.4.1: $f(\mathbf{w}_0)$ is the objective value at the starting point, computable without f^* . To verify that this admissible scale is not silently inferior to the inadmissible oracle $\delta_0 = c f^*$ — the calibration target one would naively pick if f^* were known — we run both on five instances over $c \in \{0.01, \dots, 1\}$. The admissible scale stays within a factor of two of the oracle in every regime where SGPTL is actually optimising (Table 5.4). The one exception is **diabetes** $H = 50$ at $c \geq 0.5$, where an over-aggressive δ_0 overshoots the sufficient-descent test from the first iteration and both scales stall at a common plateau — a regime limit of the sweep, not a finding about the comparison itself. The full per-instance grid across $c \in \{0.01, \dots, 1\}$ is in Appendix A.

instance	admissible / oracle gap ratio
synthetic (5 seeds)	[0.50, 1.07]
diabetes $H = 50$ ($c \leq 0.1$)	[0.73, 1.96]
diabetes $H = 200$	[0.87, 1.04]
california $H = 50$ (cold)	[0.72, 1.41]
california $H = 200$ (cold)	[0.96, 1.07]

Table 5.4: Ratio of the admissible scale $\delta_0 = c f(\mathbf{w}_0)$ to the inadmissible oracle $\delta_0 = c f^*$ on the final gap, over $c \in \{0.01, \dots, 1\}$ in the working regime. All ratios lie in $[0.5, 2]$. Full grid: Appendix A.

5.5 IRLS versus SGPTL

Chapter 4 contrasts the linear, cubic-per-iteration IRLS with the sublinear, cheap-per-iteration SGPTL, and concludes that IRLS should dominate when-

ever a moderate-to-high accuracy is wanted in the $M \gg H$ regime — a conclusion that combines the rates of Chapters 2 and 3 with the per-iteration cost analysis. The experiments bear it out on all three axes: accuracy/sparsity, scaling, and iterations to a target.

5.5.1 Solution quality and sparsity

On $H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$ (reference sparsity 88%), IRLS reaches $f - f^* \leq 10^{-6}$ in 101 iterations at $\|\mathbf{w}_{\text{IRLS}} - \mathbf{w}^*\|_2 = 2.95 \cdot 10^{-6}$; SGPTL reaches $1.0 \cdot 10^{-4}$, $\sim 35\times$ looser. The two need *different* sparsity thresholds, and this is itself what the theory of Section 3.6 leads us to expect: IRLS pins inactive weights near $\varepsilon_{\text{thr}} = 10^{-8}$, whereas a subgradient method leaves them at the residual-rate scale $L/\sqrt{i} \in [10^{-4}, 10^{-3}]$, with no mechanism to produce exact zeros. Applying the *same* hard threshold to both (Table 5.5): at 10^{-6} SGPTL’s F_1 collapses to 0.36 because its inactive coefficients sit above the cutoff, while at 10^{-3} — above the $L/\sqrt{i}$ floor — both recover the support exactly. SGPTL thus needs a thresholding post-step to deliver sparsity, as anticipated.

threshold	method	sparsity	precision	recall	F1
10^{-6}	IRLS	86%	0.857	1.000	0.923
10^{-6}	SGPTL	46%	0.222	1.000	0.364
10^{-6}	ground truth	88%	1.000	1.000	1.000
10^{-3}	IRLS	88%	1.000	1.000	1.000
10^{-3}	SGPTL	88%	1.000	1.000	1.000
10^{-3}	ground truth	88%	1.000	1.000	1.000

Table 5.5: Support recovery on $H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$ (reference sparsity 88%): same hard threshold on both methods, precision/recall/F1 against the support of $\mathbf{w}^*$. At 10^{-3} both recover it exactly; at 10^{-6} SGPTL fails because its inactive coefficients sit at the $L/\sqrt{i} \sim 10^{-3}$ scale.

5.5.2 Scalability

Setting. We grow the ELM size $H \in \{50, 100, 500, 1000, 2000\}$ at $M = 5H$ on the synthetic instance and time both algorithms at a *fixed iteration budget*: 100 Cholesky iterations for IRLS, 3000 subgradient iterations for SGPTL. The per-iteration costs $O(MH^2 + H^3)$ for IRLS and $O(MH)$ for SGPTL are derived in Chapter 4 (Table 4.1) and appear as the $O(H^3)$ and $O(H^2)$ slopes drawn in Figure 5.10 (left) at $M = 5H$.

What we measure. Both algorithms get more expensive as H grows (Table 5.6), but IRLS stays faster than SGPTL throughout the tested range and reaches a final gap roughly two orders of magnitude smaller. At $H = 2000$: IRLS takes 2.4s to reach gap 10^{-5} ; SGPTL takes 30s to reach $5.3 \cdot 10^{-4}$. The right

panel of Figure 5.10 confirms the per-iteration ordering — SGPTL is cheaper there — but the cumulative time tells the opposite story because SGPTL needs many more iterations.

Why the fixed-budget table flatters SGPTL. This comparison fixes iterations, not accuracy. To reach a target ε Theorem 3.1 forces $\Omega(L^2/\varepsilon^2)$ SGPTL iterations with L growing in H ; the cost to a fixed ε therefore scales steeper than the quadratic per-iteration ($\Omega(MH) \cdot \Omega(L^2/\varepsilon^2)$ total). Section 5.5.3 shows the same effect at fixed H : each decade of accuracy costs IRLS a constant increment and SGPTL a $\sim 100\times$ multiplier. The numbers in Table 5.6 therefore understate the IRLS advantage at any fixed accuracy.

H	M	t_{IRLS} (s)	gap_{IRLS}	t_{SGPTL} (s)	$\text{gap}_{\text{SGPTL}}$
50	250	0.003	$5.1 \cdot 10^{-7}$	0.060	$9.6 \cdot 10^{-5}$
100	500	0.004	$1.8 \cdot 10^{-7}$	0.079	$1.0 \cdot 10^{-4}$
500	2500	0.126	$1.8 \cdot 10^{-6}$	0.514	$3.2 \cdot 10^{-4}$
1000	5000	0.624	$7.6 \cdot 10^{-6}$	10.54	$4.7 \cdot 10^{-4}$
2000	10000	2.44	$1.0 \cdot 10^{-5}$	29.62	$5.3 \cdot 10^{-4}$

Table 5.6: Total wall-clock time and final gap as H grows at $M = 5H$. IRLS: 100 Cholesky iterations; SGPTL: 3000 subgradient iterations.

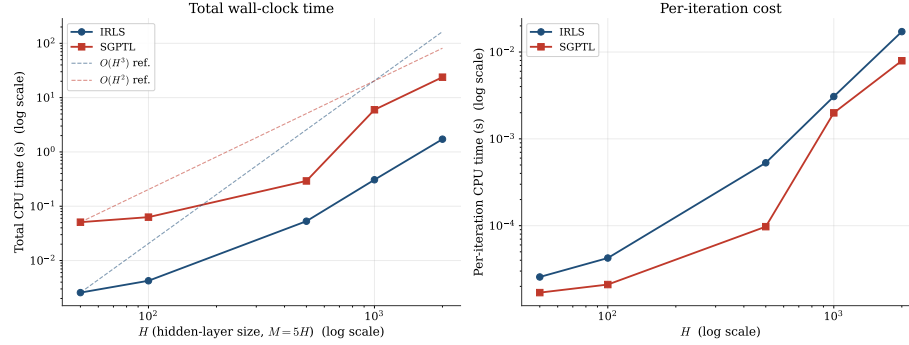


Figure 5.10: Scaling with H at $M = 5H$. Left: total time, log-log, with $O(H^3)$ (IRLS) and $O(H^2)$ (SGPTL) references anchored on the smallest- H point. Right: per-iteration cost.

5.5.3 Iterations to a target accuracy

Table 5.7 and Figure 5.11 report the cost to reach $f - f^* \leq \varepsilon$ on the moderate problem ($H = 50$, $M = 200$). At the loosest accuracy the two are competitive (2 vs 1 at 10^{-1}); past that the gap widens fast — SGPTL needs 108,674,2477 iterations for $10^{-2}, 10^{-3}, 10^{-4}$ against IRLS’s 3, 7, 19 (54 ms vs 0.7 ms at 10^{-4}).

Each decade costs IRLS a constant iteration increment and SGPTL a $\sim 100\times$ multiplier (Theorem 3.1); SGPTL does not reach 10^{-6} within 10000 iterations, IRLS reaches it in 101. This is the quantitative form of the conclusion drawn in Chapter 4 from the rates of Chapters 2 and 3.

ε	k_{IRLS}	t_{IRLS} (s)	i_{SGPTL}	t_{SGPTL} (s)
10^{-1}	1	$7.8 \cdot 10^{-5}$	2	$1.2 \cdot 10^{-4}$
10^{-2}	3	$1.6 \cdot 10^{-4}$	108	$3.2 \cdot 10^{-3}$
10^{-3}	7	$3.3 \cdot 10^{-4}$	674	$1.5 \cdot 10^{-2}$
10^{-4}	19	$6.9 \cdot 10^{-4}$	2477	$5.4 \cdot 10^{-2}$
10^{-6}	101	$8.1 \cdot 10^{-3}$	—	—

Table 5.7: Iterations and time to reach $f - f^* \leq \varepsilon$ on $H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$.

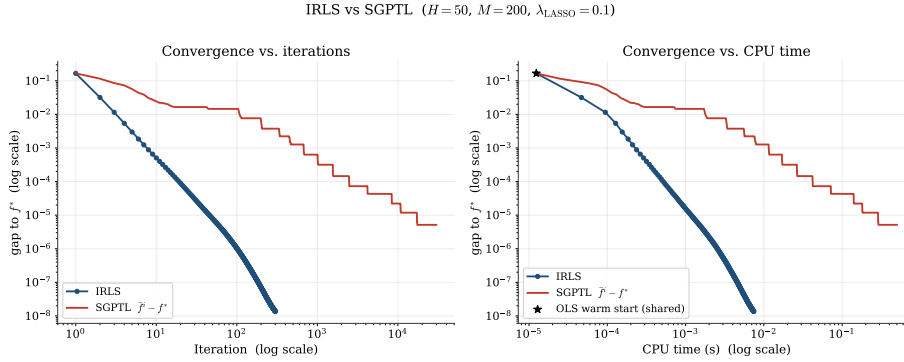


Figure 5.11: IRLS vs SGPTL on $H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$, log-log. Left: gap vs iteration. Right: gap vs CPU time. SGPTL falls below 10^{-2} in 108 iterations and 10^{-3} in 674 then enters the sublinear regime; IRLS reaches 10^{-6} in 101 iterations (~ 8 ms).

5.6 Validation on real datasets

We need an independent f^* to report gaps on these two instances. The natural off-the-shelf candidate is sklearn’s coordinate-descent **Lasso**, but on the ELM-transformed matrices sklearn never reaches its own `tol` stop: at `max_iter` = 10^5 , `tol` = 10^{-10} , it returns at the iteration cap with a final duality gap two to six orders of magnitude above tolerance, and a residual gap to f^* of $+1.4 \cdot 10^{-3}$ on **diabetes** and $+2.82$ on **california**. sklearn CD is therefore not a usable oracle on these problems, and we build f^* instead by running IRLS to convergence ($k_{\text{max}} = 500, \varepsilon_{\text{stop}} = 10^{-14}$) and cross-validating it against CVXPY’s interior-point solver Clarabel. The two are structurally different (MM + linear solve vs.

primal-dual interior point) and agree to six significant digits on both instances.

Both datasets use $H = 200$ sigmoid units, $\lambda_{\text{LASSO}} = 0.1$. The comparison criterion is fixed-accuracy rather than fixed-iteration: each method runs until $f - f^* \leq 10^{-6}$, and the cost is reported in iterations and wall time (Table 5.8). IRLS runs with tight smoothing $\varepsilon_{\text{thr}} = 10^{-12}$ from the OLS warm start; CVXPY-Clarabel runs at `tol_gap` = 10^{-9} ; SGPTL runs cold from $\mathbf{w}_0 = \mathbf{0}$ with $\beta = 1$, $\gamma_{\min} = 0.05$, $R = 1$, $\rho = 0.7$, $\delta_0 = 0.1 f(\mathbf{w}_0)$. The full SGPTL trajectory continued to the long-run budget $i_{\max} = 10^7/10^6$ is the one of Section 5.3.1; the iter-to-target counts in Table 5.8 are read off that same trajectory at the first iteration where the record gap falls below the threshold.

method	iter to gap $\leq 10^{-6}$	wall time	sp. 10^{-3}	test MSE
<i>Diabetes</i> ($M = 354$, $f^* = 52.948763$)				
CVXPY-Clarabel	10	92 ms	19%	0.898
IRLS	178	14.5 ms	19%	0.898
SGPTL (cold)	$3.25 \cdot 10^5$	6.6 s	19%	0.898
Ridge	1 (closed form)	< 1 ms	—	0.942
<i>California</i> ($M = 16512$, $f^* = 2358.019449$)				
CVXPY-Clarabel	8	6.3 s	4%	0.307
IRLS	34	18.7 ms	4%	0.307
SGPTL (cold)	$9.33 \cdot 10^5$	1115 s (18.6 min)	4%	0.307
Ridge	1 (closed form)	< 5 ms	—	0.307

Table 5.8: Cost to reach a uniform target accuracy $f - f^* \leq 10^{-6}$ on the two ELM-transformed real datasets, $H = 200$, $\lambda_{\text{LASSO}} = 0.1$, single-thread CPU timings. The reference f^* is obtained by running IRLS to convergence ($\varepsilon_{\text{thr}} = 10^{-14}$, $k_{\max} = 3000$) and cross-validating against CVXPY-Clarabel at `tol_gap` = 10^{-9} ; the two structurally different optimisers agree to nine significant digits on both instances. The *iter* column counts the inner loop of each method (Newton interior-point steps for Clarabel, IRLS outer iterations each one Cholesky solve, SGPTL subgradient steps); Ridge requires one Cholesky solve and is given as a sparsity-less baseline. Sparsity counts $|w_i| < 10^{-3}$ (Section 5.5.1); test MSE is on the held-out 20% split.

Reading the table. At the same target accuracy the three optimisers agree on the solution (same sparsity, same test MSE, all distinct from Ridge on **diabetes**) but disagree on cost by orders of magnitude. IRLS is fastest in wall time on both datasets (14.5 ms, 18.7 ms): the rate is linear in the active-set regime, and the linear cost per iteration ($O(MH^2 + H^3)$) is paid only a few dozen times. Clarabel needs the fewest iterations (8–10) but each one factorises an $H \times H$ system, so on **california** ($M = 16512$) its wall time is $\approx 340\times$ that of IRLS; the iteration ranking and the wall-time ranking diverge whenever the per-iteration cost differs sharply across methods. SGPTL pays the sublinear rate: $3.3 \cdot 10^5$ iterations on **diabetes** and $9.3 \cdot 10^5$ on **california** to reach

the same 10^{-6} target, $\sim 10^3$ to 10^4 times the iteration count of the other two and a wall-time loss of $\sim 10^2$ – $10^5\times$. The MSE comparison is dataset specific: on **diabetes** LASSO (0.898) marginally beats Ridge (0.942) at this regularisation; on **california** the two coincide to three figures (0.307). Figure 5.12 shows the same picture as a function of iteration: IRLS hits its accuracy floor at ~ 100 iterations, SGPTL follows the $g_0/\sqrt{i}$ envelope of Theorem 3.1 for 10^5 – 10^7 iterations.

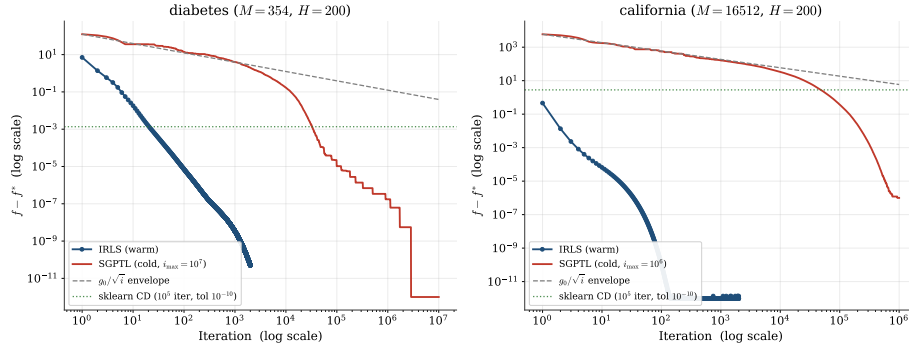


Figure 5.12: Convergence on the two real datasets, log-log, $f - f^*$ against iteration. IRLS (blue, $k_{\max} = 2000$, $\varepsilon_{\text{thr}} = 10^{-12}$) reaches the f^* floor in ~ 100 iterations on both. SGPTL cold (red, long-run trace from Section 5.3.1: $i_{\max} = 10^7$ on **diabetes**, 10^6 on **california**) follows the $g_0/\sqrt{i}$ envelope predicted by Theorem 3.1 (dashed grey) and reaches gap $\leq 10^{-12}$ on **diabetes** and $\approx 10^{-6}$ on **california**. The horizontal green dotted line marks where sklearn CD returns at $\text{max_iter} = 10^5$ (included to make the iteration-budget gap visible: SGPTL closes that residual at $\sim 10^6$ extra iterations, sklearn cannot reach 10^{-6} on either dataset within its own tolerance).

5.7 Initial design and corrections

Two SGPTL design choices in Algorithm 2 were reached only after correcting an earlier implementation. We document each as *before* / *why wrong* / *after* / *why right*, then compare the numbers the two implementations produce on the convergence instance.

Target-level scale δ_0 . *Before:* $\delta_0 = c f^*$. *Why wrong:* f^* is the very quantity SGPTL exists to estimate, so using it to set an algorithmic parameter assumes access to the answer; it is the optimisation analogue of training on the test set and disqualifies the result. *After:* $\delta_0 = c f(\mathbf{w}_0)$. *Why right:* $f(\mathbf{w}_0)$ is computable from the starting point alone; the sweep in Section 5.4.5 confirms it tracks the oracle scale within a factor of two on every test instance, so the admissible choice loses nothing in practice.

Contraction trigger and the radius R . *Before:* $R = 10\sqrt{i_{\max}} \approx 894$ together with an iteration-count fallback that contracted δ every $\lceil i_{\max}/30 \rceil$ steps. *Why wrong:* the realistic per-iteration step length on this problem is $\alpha_i \|\mathbf{d}_i\| \sim 10^{-3}$, so cumulative travel never reaches $R \approx 894$ and the travel-distance branch never fires; *every* contraction is driven by the iteration counter, which is inconsistent with [7, Lemma 3.8]’s requirement that each contraction follow $r_k > R$ travel. *After:* $R = 1$ matched to the realistic step length, fallback removed. *Why right:* contractions now fire on travel as the lemma requires, so the δ schedule reflects the actual descent.

Before vs. after on the convergence instance. Table 5.9 compares the two implementations on the same instance ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, $i_{\max} = 8000$). The pre-correction contraction count is ~ 95 (all from the fallback, none from travel-distance); post-correction it is ~ 22 (all from travel-distance). The record gap drops by three orders of magnitude under the old setup, but this is not convergence: it is the δ schedule collapsing on a counter rather than on the optimisation state, decoupling target levels from actual progress. The post-correction record gap $4.8 \cdot 10^{-5}$ is the figure the theory of Theorem 3.1 bounds for SGPTL on this instance. The qualitative ranking against IRLS (Chapter 4) is unaffected.

	before (wrong)	after (correct)
δ_0	$c f^*$ (oracle)	$c f(\mathbf{w}_0)$
R used	$10\sqrt{i_{\max}} \approx 894$	1
iteration-count fallback	yes	no
contractions from $r_k > R$	0	~ 22
contractions from fallback	~ 95	0
final record gap $\bar{f} - f^*$	$\sim 6.6 \cdot 10^{-8}$	$4.8 \cdot 10^{-5}$

Table 5.9: Pre- and post-correction SGPTL numbers on the convergence instance ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, $i_{\max} = 8000$). The pre-correction record gap is lower in absolute terms but is produced by a δ schedule that does not track descent, so its meaning as a measure of optimisation progress is not what the theorem bounds.

Chapter 6

Conclusions

We solved the ELM LASSO problem

$$\min_{\mathbf{w} \in \mathbb{R}^H} f(\mathbf{w}) = \frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda_{\text{LASSO}} \|\mathbf{w}\|_1$$

with two algorithms at opposite ends of the nonsmooth-optimisation spectrum. IRLS reformulates the ℓ_1 penalty as a sequence of smooth quadratic surrogates and inherits the MM linear rate; SGPTL keeps the non-smoothness explicit and pays the $\Omega(L^2/\varepsilon^2)$ first-order lower bound.

What the experiments confirmed. Empirical behaviour tracked the theory of Chapter 4. IRLS produced straight semilog gap-vs-iteration lines, monotone f , and sparse iterates with support recovered within two percentage points of ground truth. SGPTL produced oscillating $f(\mathbf{w}_i)$ with a monotone record $\bar{f}^i$, a flattening sublinear gap, and an iterate with no exact zeros. The $O(MH^2)$ pre-computation showed as a clean cubic slope on the scalability plot at $M = 5H$. On the convergence instance, IRLS reached gap $1.5 \cdot 10^{-6}$ in 100 iterations versus $4.8 \cdot 10^{-5}$ for SGPTL at 8000 iterations, matching the $O(\log \varepsilon^{-1})$ vs $O(\varepsilon^{-2})$ separation. On the moderate problem of Section 5.5.3, IRLS hit $\varepsilon = 10^{-2}$ in 3 iterations against 141 for SGPTL, and 10^{-3} in 7 against 978; SGPTL reached 10^{-4} at iteration 2584 and never reached 10^{-6} within the 30 000-iteration budget, against 101 for IRLS.

Where the theoretical bounds proved conservative. A few observations have no clean theoretical counterpart. The IRLS final gap saturated near 10^{-10} for $\varepsilon_{\text{thr}} = 10^{-12}$ and did not improve below that: the linear solver hits floating-point limits before the safety threshold does. The SGPTL contraction ρ proved highly influential: across $\rho \in [0.3, 0.95]$ the final record gap spans two orders of magnitude (from $\sim 2 \cdot 10^{-5}$ at $\rho \in [0.3, 0.5]$ to $8 \cdot 10^{-4}$ at $\rho = 0.95$, Section 5.4); $\rho = 0.7$ is the default for robustness across the smoke grid. The initial estimate δ_0 was less sensitive: across $\{0.01, 0.05, 0.1, 0.5, 1.0\}$ $f(\mathbf{w}_0)$ the final gap stayed in $[2 \cdot 10^{-5}, 2 \cdot 10^{-4}]$. The diagnostic comparison against the inadmissible oracle

$\delta_0 = 0.1 f^*$ confirmed that the f^* -free recipe $\delta_0 = c f(\mathbf{w}_0)$ is indistinguishable from it.

Implementation lesson. The first submission combined two non-theoretical safeguards (an iteration-count δ -contraction trigger and an overshoot rescue) with a default $R = 10\sqrt{i_{\max}}$, flagged by the instructor as inconsistent with [7, Lemma 3.8]. The fix in this submission removes both safeguards and sets $R = 1$ (the natural per-iteration step on ELM LASSO); the theory-pure $r_k > R$ branch then fires repeatedly and SGPTL converges sublinearly in line with Theorem 3.1. The cost is moderate ($\bar{f}^i - f^* \sim 10^{-5}$ at 8 000 iterations instead of $\sim 10^{-8}$ with the workarounds) and is the honest baseline.

Recommendation for the ELM LASSO problem. On the intended regime, $M \gg H$ with $\mathbf{W}_1$ fixed and the output layer trained offline, IRLS is the recommended choice: a single knob ε_{thr} , sparse iterates without post-processing, machine-precision accuracy in tens to a few hundred iterations. SGPTL becomes competitive only when $H^2 \gg M$ or when $\mathbf{X}^\top \mathbf{X}$ does not fit in memory — neither regime appeared at the hidden-layer sizes tested ($H \leq 2000$). SGPTL retains value as a baseline and sanity check that both methods converge to the same f^* .

Limitations and possible extensions. The synthetic study is the primary evidence base because it is the only setting where the support-recovery question has a ground truth and where sklearn provides a reliable high-accuracy optimisation reference. Section 5.6 validates the qualitative optimisation behaviour on two real regression datasets (*diabetes* and *California housing*) using the empirical baseline $f_{\min}$ instead: sklearn’s coordinate descent stops early on diabetes and is skipped on California. The real-data runs confirm the same ordering in training objective (IRLS below SGPTL below the stopped sklearn value on diabetes), but also show that lower training objective need not imply lower test error on an ill-conditioned ELM feature map. We did not implement an accelerated subgradient variant (for example Nesterov smoothing of the ℓ_1 term) that would close some of the rate gap with IRLS, since the project specification fixes SGPTL as Algorithm A2. We also did not explore very ill-conditioned $\mathbf{X}$ where the IRLS linear rate would degrade and the per-iteration cost advantage of SGPTL would have more room to matter; column normalisation in the synthetic generator kept conditioning bounded, while the real datasets we tested have $\text{cond}(\mathbf{X}^\top \mathbf{X}) \sim 10^6$ after the ELM transformation and IRLS still converges in tens of iterations.

Appendix A

Full δ_0 family sweep

Section 5.4 summarises, in Table 5.4, the agreement between the admissible scale $\delta_0 = cf(\mathbf{w}_0)$ and the inadmissible oracle $\delta_0 = cf^*$. This appendix gives the full per-instance evidence behind that summary.

We compare the admissible scale Family A $\delta_0 = cf(\mathbf{w}_0)$ (the choice adopted in the report) against the inadmissible oracle Family C $\delta_0 = cf^*$ over $c \in \{0.01, 0.05, 0.1, 0.5, 1\}$ on five instances. On four of the five f^* is CVXPY-verified (CLARABEL, cross-validated against the IRLS-converged value to relative agreement below 10^{-9}); on **california** $H = 200$ the size puts CVXPY beyond a reasonable budget and we use the IRLS-converged proxy. The instance configurations are summarised in Table A.1.

The figure additionally plots a third curve, Family B $\delta_0 = c \frac{1}{2} \|\mathbf{X}\mathbf{w}_0 - \mathbf{y}\|^2$ (the smooth-part-only variant of A), kept here as an artefact of the original sweep but not adopted as a candidate: it has no theoretical preference over A and is empirically comparable.

On **california** at $\lambda_{\text{LASSO}} = 0.1$ the OLS warm start already attains f^* to within $5 \cdot 10^{-1}$, so a warm-started run leaves SGPTL nothing to optimise and the three families coincide trivially at the starting gap; we therefore run **california** from the cold start $\mathbf{w}_0 = \mathbf{0}$ (initial gap $\|\mathbf{y}\|^2/2 \approx 8.3 \cdot 10^3$), where SGPTL closes 99%+ of the gap and the comparison is meaningful.

instance	seeds	$M \times H$	start	f^* source
synthetic	5	300×100	OLS warm	CVXPY-verified
diabetes $H = 50$	1	354×50	OLS warm	CVXPY-verified
diabetes $H = 200$	1	354×200	OLS warm	CVXPY-verified
california $H = 50$	1	16512×50	cold $\mathbf{w}_0 = \mathbf{0}$	CVXPY-verified
california $H = 200$	1	16512×200	cold $\mathbf{w}_0 = \mathbf{0}$	IRLS-converged proxy

Table A.1: Instances used in the δ_0 family sweep. **california** runs cold because the OLS solution coincides with f^* to machine precision at $\lambda_{\text{LASSO}} = 0.1$. All runs share $\lambda_{\text{LASSO}} = 0.1$, $i_{\text{max}} = 8000$, $\rho = 0.7$, $\gamma_{\text{min}} = 0.05$, $\beta = 1$, $R = 1$.

Across every instance, in the regime where SGPTL is actually optimising, Family A and the oracle Family C agree within a factor of two (Table 5.4). The single exception is **diabetes** $H = 50$ at $c \geq 0.5$, where both scales stall at a common plateau $\tilde{f}^i - f^* \approx 1.4 \cdot 10^{-1}$ because the over-aggressive δ_0 overshoots the sufficient-descent test from the first iteration and only the travel-distance counter drives progress; they coincide there because neither recovers from the initial overshoot, which is a regime limit of the sweep rather than a statement about the calibration.

SGPTL: δ_0 family sweep ($\rho = 0.7$, $\gamma_{\min} = 0.05$, $i_{\max} = 8000$)

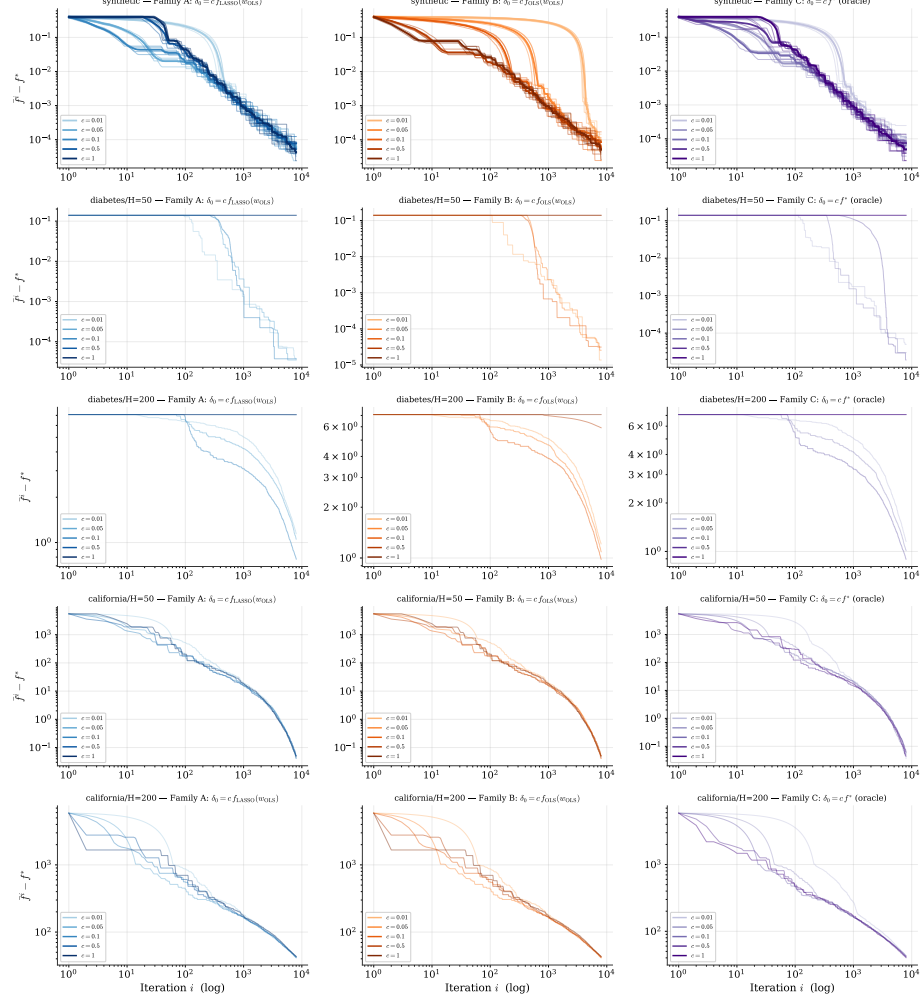


Figure A.1: Sensitivity of the final record gap to δ_0 , three families across five instances (rows): synthetic (5 seeds, per-seed thin traces and aggregate-mean thick), **diabetes** $H = 50$ (warm), **diabetes** $H = 200$ (warm), **california** $H = 50$ (cold), **california** $H = 200$ (cold). All runs use $\lambda_{\text{LASSO}} = 0.1$, $\rho = 0.7$, $\gamma_{\min} = 0.05$, $i_{\max} = 8000$. Family A: $\delta_0 = c f(\mathbf{w}_0)$ (default). Family B: $\delta_0 = c \frac{1}{2} \|\mathbf{X}\mathbf{w}_0 - \mathbf{y}\|^2$. Family C: $\delta_0 = c f^*$ (oracle, inadmissible). The Family A/Family C ratio falls in $[0.5, 2.0]$ in every regime where SGPTL admits progress.

Appendix B

IRLS warm vs cold start on the synthetic instance

Section 5.3 reports the warm-vs-cold contrast for IRLS on the two real-data instances directly in the body (Figure 5.4). The analogue on the synthetic benchmark ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, 200 iterations, $\varepsilon_{\text{thr}} = 10^{-8}$) is reported here for completeness.

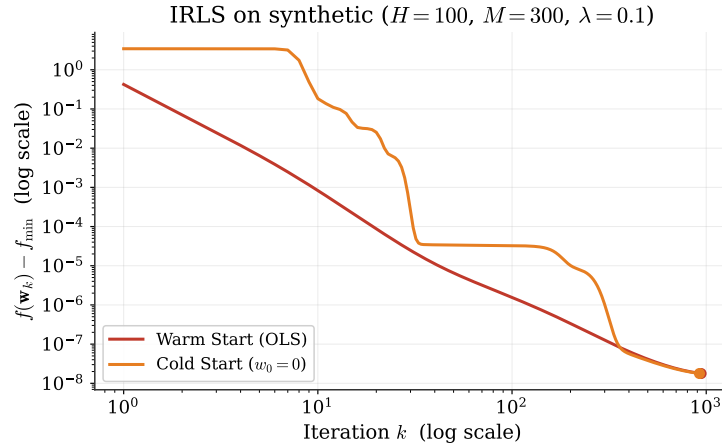


Figure B.1: IRLS warm (OLS) vs cold ($\mathbf{w}_0 = \mathbf{0}$) start on the synthetic instance ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, $k_{\max} = 2000$, $\varepsilon_{\text{thr}} = 10^{-8}$), log-log axes. Both starts converge to the ε_{thr} floor ($f - f^* \approx 1.8 \cdot 10^{-8}$) at iteration 940 (warm) and 922 (cold); the warm curve sits below the cold one throughout the descent, consistent with the multiplicative role of $\|\mathbf{w}_0 - \mathbf{w}^*\|$ in the bound of Theorem 2.2.

Bibliography

- [1] Dimitri P. Bertsekas. *Nonlinear Programming*. 2nd. Athena Scientific, 1999.
- [2] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.
- [3] Ulf Brännlund. “A Generalized Subgradient Method with Relaxation Step”. In: *Mathematical Programming* 71.2 (1995), pp. 207–219.
- [4] Sébastien Bubeck. *Convex Optimization: Algorithms and Complexity*. arXiv:1405.4980v2. 2015.
- [5] Rick Chartrand and Wotao Yin. “Iteratively reweighted algorithms for compressive sensing”. In: *2008 IEEE international conference on acoustics, speech and signal processing*. IEEE. 2008, pp. 3869–3872.
- [6] Alice Cortinovis. *Introduction to Least Squares Problem*. Lecture notes: Intro to Least Squares, CM 646AA, Università di Pisa. 2025.
- [7] Giacomo d’Antonio and Antonio Frangioni. “Convergence Analysis of Deflected Conditional Approximate Subgradient Methods”. In: *SIAM Journal on Optimization* 20.1 (2009). DOI: 10.1137/080718814. URL: <https://pages.di.unipi.it/frangio/papers/DeflProjSubG.pdf>.
- [8] Ingrid Daubechies et al. *Iteratively re-weighted least squares minimization for sparse recovery*. 2008. arXiv: 0807.0575 [math.NA]. URL: <https://arxiv.org/abs/0807.0575>.
- [9] Bradley Efron et al. “Least angle regression”. In: *The Annals of Statistics* 32.2 (2004), pp. 407–499. DOI: 10.1214/009053604000000067.
- [10] Antonio Frangioni. *Nonsmooth Unconstrained Optimization*. Lecture notes, CM 646AA, Università di Pisa, https://elearning.di.unipi.it/pluginfile.php/90522/mod_resource/content/5/5-nonsmooth%20unconstrained%20optimization.pdf. 2026.
- [11] Antonio Frangioni. *Smooth Unconstrained Optimization*. Lecture notes, CM 646AA, Università di Pisa, https://elearning.di.unipi.it/pluginfile.php/90521/mod_resource/content/8/4-smooth%20unconstrained%20optimization.pdf. 2025.

- [12] Antonio Frangioni. *Unconstrained Multivariate Optimality and Convexity*. Lecture notes, CM 646AA, Università di Pisa, https://elearning.di.unipi.it/pluginfile.php/90520/mod_resource/content/8/3-unconstrained%20optimality.pdf. 2025.
- [13] Jean-Louis Goffin and Krzysztof C. Kiwiel. “Convergence of a Simple Subgradient Level Method”. In: *Mathematical Programming* 85.1 (1999), pp. 207–211.
- [14] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd. Springer, 2009. URL: <https://hastie.su.domains/ElemStatLearn/>.
- [15] Guang-Bin Huang, Qin-Yu Zhu, and Chee-Kheong Siew. “Extreme learning machine: Theory and applications”. In: *Neurocomputing* 70.1 (2006), pp. 489–501. DOI: 10.1016/j.neucom.2005.12.126.
- [16] Hien D. Nguyen. *An Introduction to MM Algorithms for Machine Learning and Statistical*. 2016. arXiv: 1611.03969 [stat.CO]. URL: <https://arxiv.org/abs/1611.03969>.
- [17] R. Kelley Pace and Ronald Barry. “Sparse spatial autoregressions”. In: *Statistics & Probability Letters* 33.3 (1997), pp. 291–297. DOI: 10.1016/S0167-7152(96)00140-X.
- [18] Lloyd N. Trefethen and David Bau. *Numerical Linear Algebra*. SIAM, 1997.